



CU Online

Discover. Learn. Empower.



Project Title: AI Stock Report Generator: An Agentic System for Automated Equity Research Report Generation

Submitted in partial fulfillment of the requirements for the award of the
degree of

Master of Science (Data Science)

By

Student Name: Sulaiman Mohammadyasin Mutawalli

Enrollment No: O24MSD110357

Under the guidance of

Guide/Mentor Name: Deep Ghosh

Declaration

I, Sulaiman Mohammadyasin Mutawalli, hereby solemnly declare that the project report titled "**AI Stock Report Generator: An Agentic System for Automated Equity Research Report Generation**" submitted in partial fulfillment of the requirements for the award of the degree of **Master of Science (Data Science) MSc (DS)** is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2025-2026 under the supervision of **Deep Ghosh**.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: Bengaluru

Date: 30 May 2026

Student Signature: Sulaiman Mutawalli

Student Name: Sulaiman Mutawalli

Enrollment No: O24MSD110357

Abstract

This project presents an agentic AI stock report generator that transforms a natural-language user request into a structured, evidence-backed equity research PDF. The objective was to reduce manual effort in stock analysis by integrating ticker resolution, multi-source data retrieval, chart generation, AI-assisted narrative writing, and downloadable report delivery into a single workflow. The system was implemented using Python 3.12 with FastAPI for backend services, Streamlit for the user interface, Google Gemini for controlled language-generation tasks, yfinance/feedparser/SEC EDGAR data sources for evidence gathering, Matplotlib for chart creation, and ReportLab for PDF assembly. An asynchronous orchestrator coordinates research and report-generation stages while server-sent events provide live progress feedback to the user.

The development followed an iterative, implementation-led approach aligned with Agile principles. The final system includes modular services, containerized execution support, and deployment-oriented artifacts. Validation was performed through a completed pytest suite covering unit and integration levels, with all tests passing in the validated run. Results indicate that the system can successfully generate coherent report artifacts with narrative and visual evidence while improving workflow transparency and reducing fragmentation compared with manual approaches. The project demonstrates a practical and scalable foundation for future enhancements such as persistent storage, stronger access control, formal performance benchmarking, richer UI automation, and extended cloud production architecture.

Table of Contents

Declaration.....	2
Abstract.....	3
Table of Contents.....	4
Chapter 1: Introduction.....	9
1.1 Background of the Project.....	9
1.2 Problem Statement.....	10
1.3 Objectives of the System.....	10
1.4 Scope of the Project.....	11
1.5 Existing System Overview.....	12
1.6 Proposed System Overview.....	13
1.7 Technologies Used.....	14
1.8 Chapter Summary.....	14
Chapter 2: Literature Review / System Study.....	16
2.1 Introduction.....	16
2.2 Review of Similar Systems.....	17
2.2.1 Bloomberg Terminal.....	17
2.2.2 Yahoo Finance.....	17
2.2.3 OpenBB.....	18
2.2.4 General-Purpose AI Assistants.....	19
2.2.5 Partial Solutions in the Financial Tool Ecosystem.....	19
2.3 Comparative Analysis.....	20
2.3.1 Comparison Dimensions.....	20
2.3.2 Implications for the Current Design.....	21
2.4 Software Development Models.....	21
2.4.1 Waterfall Model.....	21
2.4.2 Agile Model.....	22
2.4.3 Why Agile Fits This Project.....	22
2.5 Relevant Frameworks and Libraries.....	22
2.5.1 FastAPI.....	23
2.5.2 Streamlit.....	23
2.5.3 Google Gemini SDK.....	24
2.5.4 yfinance.....	24
2.5.5 feedparser.....	24
2.5.6 matplotlib.....	24
2.5.7 ReportLab.....	24
2.5.8 Docker.....	25
2.5.9 pytest and Related Testing Tools.....	25

2.6 Research Gap.....	25
2.7 Chapter Summary.....	26
Chapter 3: System Analysis.....	27
3.1 Introduction.....	27
3.2 Functional Requirements.....	27
3.2.1 Functional Requirement Set.....	28
3.2.2 Requirement Traceability.....	30
3.3 Non-Functional Requirements.....	30
3.3.1 Performance.....	31
3.3.2 Reliability.....	31
3.3.3 Usability.....	32
3.3.4 Security.....	32
3.3.5 Maintainability.....	32
3.3.6 Scalability.....	32
3.3.7 Portability.....	33
3.4 User Requirements.....	33
3.5 Feasibility Study.....	33
3.5.1 Technical Feasibility.....	34
3.5.2 Economic Feasibility.....	34
3.5.3 Operational Feasibility.....	35
3.6 System Architecture.....	35
3.7 Data Flow Diagrams.....	36
3.7.1 DFD Level 0.....	36
3.7.2 DFD Level 1.....	37
3.8 Use Case Diagram.....	39
3.9 Chapter Summary.....	40
Chapter 4: System Design.....	42
4.1 Introduction.....	42
4.2 Use Case Design.....	43
4.3 Class Design.....	44
4.4 Sequence Design.....	46
4.5 Activity Design.....	47
4.6 ER Diagram and Runtime Data Model.....	49
4.7 Database Schema and Table Structures.....	50
4.7.1 JobRecord Structure.....	51
4.7.2 StockResearch Structure.....	51
4.7.3 ReportArtifact Structure.....	51
4.7.4 ReportSection Structure.....	52

4.8 API Design.....	52
4.9 UI/UX Wireframe Design.....	54
4.10 Deployment and Component Design.....	55
4.11 Chapter Summary.....	56
Chapter 5: System Implementation.....	57
5.1 Introduction.....	57
5.2 Development Environment.....	58
5.3 Hardware and Software Requirements.....	59
5.4 Tools and Technologies Used.....	61
5.5 Module-wise Implementation.....	64
5.5.1 Application Entry Point and Core API Setup.....	65
5.5.2 Configuration and Schema Layer.....	65
5.5.3 API Routing and Client Interaction Surface.....	66
5.5.4 In-Memory Job Tracking.....	67
5.5.5 Orchestration Layer.....	67
5.5.6 Research Agent.....	68
5.5.7 Report Agent.....	68
5.5.8 Chart Generation Service.....	69
5.5.9 PDF Generation Service.....	70
5.5.10 Streamlit Frontend.....	71
5.6 Key Algorithms and Processing Logic.....	72
5.7 Security Features.....	73
5.8 Version Control and CI/CD.....	75
5.9 Screenshots of Application.....	76
5.10 Chapter Summary.....	77
Chapter 6: Testing and Quality Assurance.....	79
6.1 Introduction and Testing Strategy.....	79
6.2 Unit Testing.....	80
6.2.1 Job Store Testing.....	81
6.2.2 Narrative Finalization Testing.....	81
6.2.3 Chart Generation Testing.....	82
6.2.4 PDF Generation Testing.....	82
6.2.5 Ticker Resolution Testing.....	83
6.3 Integration Testing.....	84
6.4 System Testing and Manual Verification.....	85
6.5 Test Cases and Bug Reports.....	86
6.6 Testing Limitations.....	87
6.7 Chapter Summary.....	88

Chapter 7: Results and Discussion.....	90
7.1 Introduction.....	90
7.2 Output Screenshots and Observed Results.....	90
7.3 Performance Evaluation.....	92
7.4 Comparison with Existing Systems.....	94
7.5 Improvements Achieved.....	95
7.6 Discussion of Findings.....	97
7.7 Chapter Summary.....	97
Chapter 8: Conclusion and Future Enhancements.....	99
8.1 Introduction to the Final Chapter.....	99
8.2 Conclusion: Achievement of Objectives.....	99
8.3 Key Achievements.....	101
8.4 Technical Learning Outcomes.....	102
8.5 Future Enhancements.....	103
8.6 Final Closure.....	104
Chapter 9: References.....	106
Chapter 10: Appendices.....	110
Appendix A: Complete Source Code Index (Project Map).....	110
A.1 Root-Level Files.....	110
A.2 Backend Application Package (`app/`).....	110
A.3 Agents (`app/agents/`).....	110
A.4 Services (`app/services/`).....	111
A.5 Tools (`app/tools/`).....	111
A.6 Models (`app/models/`).....	111
A.7 Routers (`app/routers/`).....	111
A.8 Tests (`tests/`).....	111
Appendix B: Database / Data Storage Note and Runtime Data Model.....	112
B.1 Current Storage Approach.....	112
B.2 Why No Relational Dump Is Included.....	112
B.3 Conceptual Runtime Data Entities.....	112
Appendix C: User Manual.....	113
C.1 Purpose.....	113
C.2 User Workflow.....	113
C.3 Input Guidance.....	113
C.4 Output Interpretation.....	113
C.5 Common User Issues.....	114
Appendix D: Installation Guide.....	114
D.1 Prerequisites.....	114

D.2 Local Setup Steps.....	114
D.3 Optional Containerized Setup.....	115
D.4 Validation Checks.....	116
Appendix E: API Usage Quick Reference.....	116
E.1 Endpoints.....	116
E.2 Example Stream Request.....	116
E.3 Example Download Request.....	116
Appendix F: Sample Output and Screenshot Placeholders.....	117
Appendix G: Testing Execution Evidence (Optional Supporting Appendix).....	117
Appendix H: Submission Packaging Checklist.....	117
List of Abbreviations.....	118

Chapter 1: Introduction

1.1 Background of the Project

Financial research is one of the most information-dense activities in the investment domain. A meaningful equity analysis requires the collection of several distinct evidence types: historical market prices, company fundamentals, quarterly results, valuation measures, recent news, and regulatory filings. In traditional workflows, these inputs are spread across different websites, terminals, spreadsheets, and PDF documents. The analyst or investor must manually move between these sources, compare the information, and then turn the findings into a coherent written report. This process is time-consuming, repetitive, and difficult to standardise.

The rise of large language models has created a new opportunity in this space. Modern LLMs can understand natural-language queries, summarise structured and unstructured evidence, and generate fluent analytical text. On their own, however, they are not enough to produce a trustworthy stock report. They still need external data sources, explicit orchestration, chart generation, and a controlled output format. A useful reporting system therefore has to combine generative AI with evidence retrieval and document assembly rather than relying on text generation alone.

This project addresses that need by implementing an agentic AI system for automated equity research report generation. The user submits a plain-English request such as “Generate a report for Nvidia stock,” and the system identifies the relevant ticker, gathers market and financial data, fetches current news and SEC filings, generates analytical charts, writes report sections with Gemini, and assembles the final result into a downloadable PDF. The same request is also streamed live to the frontend so that the user can see progress as each stage completes.

The relevance of this project is both practical and academic. Practically, the system reduces the manual effort required to prepare an investment-style research report. Academically, it demonstrates a complete software engineering workflow: API design, asynchronous orchestration, external data access, prompt engineering, chart generation, PDF rendering, testing, containerisation, and deployment preparation. These elements

align closely with the expectations of an MSc (Data Science) project because they demonstrate implementation depth rather than only theory.

1.2 Problem Statement

The core problem addressed by this project is the lack of an accessible system that can transform a natural-language stock query into a professional, evidence-backed report with minimal user effort. Existing financial research processes are fragmented. A user who wants to analyse a company must first identify the correct ticker, then inspect price history, then review financial statements, then read recent news, then check filing activity, and finally write the findings into a structured report. Each of these steps adds effort, and the combined workflow is slow enough to discourage frequent use.

The problem becomes more significant when the user does not have access to expensive professional tools. Institutional financial terminals provide strong data integration, but they are not designed for the broad student or retail audience and are often inaccessible due to cost. Free market websites are available, but they still leave the user with a manual synthesis problem. General-purpose AI assistants can help with writing, but they do not automatically ground their output in structured financial evidence unless that evidence is manually supplied.

The result is a gap between data availability and report production. Data may exist, and language models may exist, but the complete workflow does not. The project therefore solves a software engineering problem rather than only a finance problem: how to connect multiple services into one repeatable pipeline that accepts a query, retrieves evidence, generates analysis, and delivers a polished artifact.

1.3 Objectives of the System

The main objective of the project is to build a working system that automates the production of an equity research report from a natural-language query. To support that objective, the system is designed with the following specific goals.

First, it must accept a plain-English stock request through a simple frontend. The user should not need to know the ticker symbol in advance. Second, it must resolve the relevant ticker and company name from the query, either through an explicit ticker or through LLM-based extraction. Third, it must gather price data, company fundamentals, valuation metrics, recent news, and SEC filing information from external sources. Fourth, it must process those data sources efficiently, using parallel asynchronous execution where possible. Fifth, it must generate charts that help explain the stock's recent performance and financial trajectory. Sixth, it must use a language model to write section-wise analytical narratives based on the gathered evidence. Seventh, it must compile the report into a professional PDF with embedded charts. Eighth, it must provide live progress updates so the user can track the pipeline while it runs. Ninth, it must expose a status endpoint and a download endpoint through a clear API layer. Tenth, it must be container-ready for local and cloud deployment. Eleventh, it must be tested so that the implementation can be verified and maintained.

These objectives are not abstract requirements. They correspond directly to the actual components in the codebase: the Streamlit UI, the FastAPI router, the orchestrator, the research agent, the report agent, the chart builder, the PDF generator, and the test suite. That traceability makes the project implementation-focused and suitable for formal academic reporting.

1.4 Scope of the Project

The scope of the project defines what the current implementation does and what it intentionally leaves out. Within scope, the system supports analysis of publicly listed stocks, natural-language query handling, optional explicit ticker input, live streaming of progress events, AI-authored report sections, embedded financial charts, PDF generation, and final artifact download. The system is built for a local development environment but also includes deployment-related configuration for containerised execution.

The project uses a FastAPI backend to expose APIs, a Streamlit frontend to collect requests and display progress, Google Gemini for ticker resolution and narrative generation, yfinance for market and financial data, Google News RSS for recent headlines, SEC EDGAR for filing metadata, matplotlib for chart creation, and ReportLab for PDF composition. The current implementation also includes a completed test suite that covers both unit and integration behaviour.

Outside the current scope are brokerage integration, trading execution, portfolio tracking, user accounts, persistent history, and relational database storage. The job store is intentionally in-memory because the project is implemented as a single-user MVP rather than a production multi-tenant financial platform. The system is also focused on report generation rather than direct investment advice or trade execution.

1.5 Existing System Overview

Several existing systems address parts of the problem, but not the complete workflow. Bloomberg Terminal is the best-known example of a professional financial research environment. It provides deep market coverage, news, analytics, and workflow tools, but it is expensive and intended for institutional use. It is an excellent benchmark for quality and completeness, but it is not a realistic baseline for an accessible MSc project.

Yahoo Finance is a more accessible alternative. It provides stock prices, company profiles, historical charts, earnings summaries, and news links. It is helpful for quick manual research, but it still expects the user to navigate between pages and mentally combine the evidence. It does not automatically create a coherent report or produce a downloadable PDF with AI-authored analysis.

OpenBB offers a different approach by providing an open-source financial analysis platform that is more technical and extensible. It is useful for programmatic workflows and shows that serious financial tooling can be open-source. However, it still does not implement the exact agentic pipeline of this project: automatic ticker resolution, parallel evidence gathering, report writing, chart generation, and PDF assembly in one request flow.

General-purpose AI assistants can also help with analysis, but they are not naturally grounded in live financial data and are not designed to produce a structured report artifact. They can describe a company, but they do not by themselves retrieve evidence, generate charts, or assemble a professionally formatted document.

Table 1.1: Comparison of existing systems

System	Cost/Access	Automation Level	AI Narrative	Report Export	Live Progress	Fit for This Project Goal
Bloomberg Terminal	High cost, enterprise access	Medium (tool-assisted)	Limited integrated narrative authoring	No direct automated custom PDF workflow	No	Benchmark only, not accessible
Yahoo Finance	Free/public	Low (mostly manual browsing)	No	No automated structured report export	No	Data source reference
OpenBB	Open-source, technical users	Medium (scriptable workflows)	Limited by user integration	Not focused on one-click report artifact	No	Useful comparison baseline
General AI Assistants	API/free-tier dependent	Medium for text only	Yes	No built-in evidence pipeline PDF	No	Partial solution only
Proposed System	Low-cost stack	High (end-to-end pipeline)	Yes (evidence-guided)	Yes (generated PDF)	Yes (SSE timeline)	Directly solves target problem

Purpose: This table should compare Bloomberg Terminal, Yahoo Finance, OpenBB, general-purpose AI assistants, and the proposed system across cost, accessibility, automation, AI synthesis, chart generation, PDF export, and live progress visibility.

1.6 Proposed System Overview

The proposed system is an end-to-end application for automated equity research report generation. The user interacts with a Streamlit interface and enters a stock-related question in natural language. The frontend sends the request to a FastAPI backend that coordinates a multi-stage pipeline through an orchestrator. The orchestrator first resolves the ticker, then invokes the research agent to gather evidence, then passes the result to the report agent, and finally sends the assembled report to the PDF generator.

A defining characteristic of the system is progress visibility. Instead of waiting silently for a long-running task to finish, the backend emits progress events at each stage of the pipeline. These events are delivered to the frontend using Server-Sent Events, which makes the user interface feel responsive and transparent. This is especially important for an AI-assisted workflow because the user needs to know whether the system is resolving the ticker, collecting data, writing the report, or producing the final PDF.

The architecture is also intentionally modular. Data acquisition is separated from report writing, and document generation is separated from the web layer. This makes the system easier to test and easier to explain in a report. It also means that individual parts can be

replaced later. For example, the in-memory job store can be replaced with Redis or another persistent store if the project is extended into a multi-user system.

The proposed system therefore combines automation, transparency, and modularity in one working application. It is not merely a chat interface. It is a controlled, pipeline-driven research tool that transforms a query into an output artifact.

1.7 Technologies Used

The implementation uses several technologies that each serve a distinct role. FastAPI provides the backend API layer because it supports asynchronous endpoints, clean request validation, and structured dependency handling. Streamlit provides the frontend because it is practical for rapidly building interactive data-focused interfaces. Google Gemini provides the generative capability required for ticker resolution and narrative writing.

For data retrieval, yfinance is used to access market prices and company-level financial information, feedparser is used to read Google News RSS feeds, and SEC EDGAR is used for filing search. For charting, matplotlib creates PNG images that can be embedded in the report. For PDF generation, ReportLab provides precise layout control and allows the system to combine text, headings, tables, and images into a professional document.

Pydantic and pydantic-settings handle schema validation and configuration management. Docker and Docker Compose support local and containerised deployment. Pytest, pytest-asyncio, pytest-mock, and pytest-cov support the testing strategy. These choices are suitable for an MSc-level implementation because they balance realism, accessibility, and technical depth.

1.8 Chapter Summary

This chapter introduced the motivation, problem, objectives, scope, and technical foundation of the project. It established that the project addresses a genuine workflow problem: the difficulty of creating a coherent stock research report from fragmented

information sources. It also explained why the chosen technologies are suitable for solving that problem in a structured, maintainable, and academically defensible way.

The next chapter reviews similar systems, software models, frameworks, and libraries in order to show how the proposed implementation fits into the broader literature and tool ecosystem.

Chapter 2: Literature Review / System Study

2.1 Introduction

The purpose of the literature review chapter is to position the proposed AI stock report generator within the wider landscape of financial research systems, generative AI tools, software development models, and the frameworks used to implement the current application. A literature review in an MSc Data Science report should not be a general technology summary. It should explain what is already available, what those systems do well, what they do not solve, and why the current implementation is a meaningful response to that gap.

This distinction is important in the present project because the application is not built around a single technology. It combines a frontend interface, an API backend, asynchronous orchestration, multi-source financial data retrieval, AI-generated text, chart creation, PDF composition, and live progress streaming. As a result, the chapter must span both domain tools and software engineering tools. It also needs to remain tied to the actual implementation rather than drifting into generic descriptions of financial markets or generic AI theory.

The best way to read this chapter is as a justification layer for the architecture and design of the system. Each system or framework discussed below is relevant because it influenced a decision in the current codebase. For example, FastAPI matters because the system needs a lightweight asynchronous API. Streamlit matters because the user interface must be simple and fast to build. yfinance matters because the project needs low-cost market data access. ReportLab matters because the final output must be a professional PDF. These are not arbitrary technology mentions; they are the software decisions that shape the implementation.

2.2 Review of Similar Systems

2.2.1 Bloomberg Terminal

Bloomberg Terminal is one of the most recognised professional financial research systems in the world. It provides market data, news, analytics, messaging, and workflow tools in a tightly integrated environment. Its key strength is completeness: a user can move from price analysis to news to valuation to communication without leaving the platform. For institutional analysts, that integrated workflow is extremely valuable because it reduces friction and supports rapid decision-making.

For this project, Bloomberg is important as a benchmark rather than a direct model. It shows what a mature financial research product can look like. However, it is also expensive, proprietary, and designed primarily for enterprise and institutional users. That makes it a poor fit for a student project that needs to demonstrate accessible engineering. More importantly, Bloomberg does not solve the exact workflow implemented in this application, which is the automatic transformation of a natural-language question into a generated equity research PDF with live progress feedback.

The difference is subtle but important. Bloomberg is a rich research environment used largely by humans to do manual analysis. The proposed system is an automation pipeline that performs several of the evidence-gathering and writing steps on behalf of the user. In the report, Bloomberg should therefore be presented as a high-end reference point that highlights the gap between premium financial terminals and accessible automated reporting systems.

2.2.2 Yahoo Finance

Yahoo Finance is a widely used free source for stock prices, company profiles, charts, news links, and selected financial information. Its popularity comes from convenience: the interface is familiar, the information is easy to access, and many casual users rely on it as a first stop for market research. It is therefore relevant to this project because it demonstrates how financially useful data can be made publicly accessible.

At the same time, Yahoo Finance still expects manual work from the user. A person must inspect a ticker page, open sections, read the articles or financial statements, and then synthesise the evidence independently. That is sufficient for quick browsing, but it is not the same as generating a professional report. There is no built-in workflow that accepts a natural-language request, resolves a ticker automatically, gathers multiple evidence sources, and compiles a structured document.

The current project uses `yfinance`, which is a Python library that provides programmatic access to Yahoo Finance-style data. This is a practical implementation choice because it lets the system retrieve prices and financial data without requiring a paid data subscription. In the context of this project, Yahoo Finance is best understood as a data source and reference platform rather than as a complete solution.

2.2.3 OpenBB

OpenBB is an open-source financial analysis platform that offers a modular environment for market research and data access. It is attractive to technically inclined users because it supports programmatic workflows and is more extensible than a typical consumer finance portal. It demonstrates that financial tooling can be open and developer-oriented rather than locked into institutional software ecosystems.

OpenBB is relevant to this project because it shows the direction of modern open financial tooling: modularity, extensibility, and programmatic access. However, it still does not provide the exact agentic report-generation pipeline implemented here. The proposed system does something more specific: it resolves a ticker from a natural-language query, gathers market/fundamental/news/filing evidence, generates narrative sections using Gemini, creates charts, and produces a PDF while streaming progress updates to the frontend.

This distinction is important for the report because it clarifies the project's contribution. The system is not simply another finance dashboard. It is an integrated AI-assisted research workflow that turns public data into a structured deliverable. OpenBB is a useful comparison point because it confirms that open-source financial analysis can be serious and extensible, but the proposed system focuses on report generation and live orchestration rather than on general-purpose trading or analytics tooling.

2.2.4 General-Purpose AI Assistants

General-purpose AI assistants such as Gemini, ChatGPT, or Claude can produce coherent explanations, summaries, and narrative text. They are useful for tasks that involve rewriting, summarisation, brainstorming, and language generation. This is why they matter to the current project: the report agent depends on model-generated section narratives that are readable, coherent, and tailored to the assembled evidence.

However, a standalone assistant does not solve the complete report workflow. If it is not connected to live data sources, its output may be generic or poorly grounded. If it is not connected to a document generator, the output remains an unstructured conversation. If it is not connected to a job tracker or streaming mechanism, the user does not know whether the task is progressing or stalled. In short, language generation alone is not the same as report generation.

The current implementation uses Gemini as one component in a larger pipeline. The model is not asked to invent facts or estimate values without context. Instead, it receives a structured research bundle that contains the relevant stock evidence, and it is asked to write specific sections using that evidence. This makes the generated report more reliable and better suited to academic and investment-style presentation.

2.2.5 Partial Solutions in the Financial Tool Ecosystem

Many other tools solve only one part of the reporting problem. Some tools provide charts but no narrative synthesis. Some provide financial data but no report writing. Some offer AI-generated summaries but do not integrate with document output. Some offer APIs but require heavy technical configuration. These partial solutions are important because they show that the problem is not a lack of individual features. The problem is the lack of a complete workflow that connects all of those features into one usable system.

The proposed application addresses this fragmentation by combining query handling, evidence gathering, chart generation, AI writing, PDF rendering, and streaming feedback.

The user does not need to manually identify the ticker, collect data from multiple websites, or assemble the final report structure. This workflow completeness is the main practical contribution of the project and the reason the literature review should be framed around integration rather than isolated functionality.

2.3 Comparative Analysis

Table 2.1 here: Comparative literature/system review

Dimension	Bloomberg Terminal	Yahoo Finance	OpenBB	General AI Assistants	Proposed System
Primary Strength	Market depth and professional tooling	Accessibility and free data visibility	Extensible open-source analytics	Fluent language generation	Integrated automated report workflow
Main Limitation	Cost and institutional orientation	Manual synthesis burden	Technical setup and non-report-first workflow	Weak grounding without structured data	MVP scope limitations
User Effort for Final Report	High	High	Medium-High	Medium-High	Low
Multi-Source Integration	Strong	Partial	Strong (technical)	External integration required	Built-in for target workflow
Suitability for Academic MVP	Low	Medium	Medium	Medium	High

Purpose: This table should compare Bloomberg Terminal, Yahoo Finance, OpenBB, general-purpose AI assistants, and the proposed system across cost, accessibility, automation, AI narrative support, chart generation, PDF export, live progress visibility, and structured multi-source integration.

2.3.1 Comparison Dimensions

A useful comparison must be based on concrete dimensions. The most relevant dimensions for this project are cost, accessibility, degree of automation, data source integration, narrative generation capability, chart generation, document export, and transparency of execution. These criteria are appropriate because they directly reflect the user experience the proposed system aims to improve.

If the comparison is evaluated against those dimensions, the limitations of existing tools become obvious. Professional terminals are highly capable but expensive. Free portals are accessible but manual. Open-source tools are flexible but often require technical setup and still do not produce polished reports automatically. General AI assistants are fluent

but weak on traceability and grounding. The proposed system seeks a balance: low-cost sources, automation, structured evidence, narrative generation, and PDF delivery.

2.3.2 Implications for the Current Design

The comparative analysis supports several choices in the implementation. Because professional systems are expensive, the project uses free or low-cost data sources. Because free portals are manual, the system automates retrieval and writing. Because AI assistants can hallucinate or remain generic without context, the system first constructs a research bundle before asking Gemini to write section narratives. Because users should be able to see progress, the system streams events through SSE. Because the result must be academic and shareable, the output is a PDF rather than a chat transcript.

This means the comparison is not decorative. It is directly connected to architecture decisions. The literature review therefore functions as a bridge between the external ecosystem and the internal implementation.

2.4 Software Development Models

2.4.1 Waterfall Model

The Waterfall model is a sequential approach in which requirements, design, implementation, testing, and deployment are treated as distinct phases. It works well when requirements are stable and well understood from the beginning. It also makes documentation straightforward because each phase can be reported separately.

For this project, however, Waterfall is not the best model to describe how the system evolved. The current implementation was refined iteratively. The report generation pipeline required adjustments. The PDF layout had to be corrected. The output filename logic had to be improved. The ticker resolution path had to be made more robust after

testing. These are not signs of a failed process; they are normal signs of iterative development. A linear Waterfall description would not accurately capture the way the project changed during implementation.

2.4.2 Agile Model

Agile development is more appropriate because it supports incremental delivery, feedback-driven refinement, and continual integration of testing results. The actual system reflects this style of work. A functional MVP came first, and then the system was expanded with better report generation, live progress streaming, Docker support, deployment configuration, and formal tests.

Agile is also a better fit for an AI-assisted system because the behaviour of prompts, the quality of model output, and the integration of external data sources can only be validated through working execution. For example, if a narrative is truncated mid-sentence, the problem is discovered by running the system rather than by reading a design document. If a PDF page layout is visually incorrect, the issue is observed at runtime. These are exactly the kinds of changes that are easier to handle iteratively than in a rigid one-pass model.

2.4.3 Why Agile Fits This Project

The report should present Agile as the development model because it reflects the real lifecycle of the current system. The project started with the core research and report workflow, then was incrementally expanded and corrected. This made it possible to refine both the user experience and the quality of the generated output. For an MSc Data Science project, that is a strong argument because it demonstrates practical software iteration rather than only theoretical planning.

2.5 Relevant Frameworks and Libraries

Table 2.2 here: Framework relevance to the proposed system

Framework/Tool	Role in System	Why Selected
FastAPI	Backend API and streaming endpoints	Async support, validation, clear route design
Streamlit	User-facing interface	Rapid development for data/AI workflows
Gemini API	Ticker resolution and section narrative generation	Strong LLM capability with API integration
yfinance	Market and fundamentals retrieval	Practical low-cost data access
feedparser	News RSS parsing	Lightweight and free news ingestion
SEC EDGAR API	Filings retrieval	Official regulatory source
Matplotlib	Chart generation	Reliable server-side plotting
ReportLab	PDF report assembly	Precise document layout control
Docker/Compose	Packaging and local orchestration	Reproducible execution
pytest stack	Verification and regression safety	Strong Python testing ecosystem

Purpose: This table should map FastAPI, Streamlit, Gemini SDK, yfinance, feedparser, matplotlib, ReportLab, Docker, and pytest to their responsibilities in the implementation.

2.5.1 FastAPI

FastAPI is the core backend framework for the application. It was selected because the project needs a lightweight, asynchronous API that can expose health, report streaming, job status, and download endpoints. Its support for request models, automatic validation, and dependency injection makes it appropriate for a pipeline-oriented service. In the current system, FastAPI is not simply a web server; it is the coordination layer that connects the frontend to the orchestrator and the artifact download path.

2.5.2 Streamlit

Streamlit is the frontend technology used to create the user-facing interface. It is suitable because the application needs an input form, a progress display, and a download button rather than a complex interactive application with many front-end routes. Streamlit allows the project to focus on the workflow itself instead of on JavaScript infrastructure. That makes it ideal for a data-heavy academic prototype.

2.5.3 Google Gemini SDK

The Gemini SDK is used for model-driven tasks such as ticker resolution and report section writing. Its role in the system is controlled rather than open-ended. The project does not treat the model as an uncontrolled chat interface. Instead, it uses the model for structured JSON generation and section-specific narrative generation. This is important because the final report must be grounded in external financial evidence rather than in free-form model speculation.

2.5.4 yfinance

The yfinance library provides access to market data and company financial information through a Python interface that is practical for the project's budget and scope. It enables the system to obtain price history, company metadata, and quarterly statements without a paid API subscription. In the implementation, it is used inside background threads so that blocking network activity does not interfere with the asynchronous request flow.

2.5.5 feedparser

feedparser is used to process Google News RSS data. This is a pragmatic choice because recent headlines are valuable in financial research and RSS is available without a news API key. By using RSS feeds, the system can gather news in a structured and lightweight manner and then include that evidence in the research bundle for narrative generation.

2.5.6 matplotlib

matplotlib is used to create the charts embedded into the PDF. The project needs image output, not interactive graphs, because the charts must be embedded into a report document. matplotlib is therefore well suited to the task. In the current application, it creates price, revenue, and margin charts as PNG bytes using a headless backend.

2.5.7 ReportLab

ReportLab is used for the final document assembly stage. It is well suited to this project because it allows direct programmatic control over page layout, fonts, tables, paragraphs, images, and page breaks. A report such as this is not a webpage; it is an academic document with cover page formatting, numbered sections, and embedded charts. ReportLab provides the level of control required for that output.

2.5.8 Docker

Docker is used to package the application into reproducible containers. This matters because the project has both an API service and a Streamlit UI, and both should behave the same in local and deployment environments. Docker also supports the report's deployment discussion by showing that the system is not only a local demo but also a container-ready application.

2.5.9 pytest and Related Testing Tools

pytest, pytest-asyncio, pytest-mock, and pytest-cov are used to verify the implementation. Their presence is important in the literature review because testing is a required part of the university report and a required part of engineering quality. The project uses these tools to validate job tracking, chart creation, PDF generation, ticker resolution, and API behaviour. This shows that the system is not merely conceptually designed but actually tested.

2.6 Research Gap

After reviewing the existing systems and supporting frameworks, the main gap becomes clear. The financial technology ecosystem contains strong individual capabilities, but those capabilities are fragmented. One system may provide data. Another may provide charts. Another may provide generative text. Another may provide deployment support. Very few systems combine all of these into one accessible, end-to-end reporting workflow.

The gap addressed by the present project is therefore not the absence of information or the absence of language models. It is the absence of a low-cost, implementation-ready, agentic pipeline that accepts a stock-related query, retrieves and organises evidence, writes structured analytical sections, generates a PDF, and streams progress while it works. That is the real contribution of the project.

The proposed application fills this gap by combining public or low-cost data sources, a controlled LLM writing layer, chart generation, live SSE updates, PDF assembly, and deployment-friendly packaging. The result is not a replacement for professional financial terminals, but a practical and academically defensible automated reporting system.

2.7 Chapter Summary

This chapter reviewed the most relevant categories of existing systems and supporting technologies. It showed that Bloomberg Terminal, Yahoo Finance, OpenBB, and general-purpose AI assistants each solve part of the financial research challenge, but none of them provide the exact end-to-end workflow implemented here. It also showed that Agile development better matches the actual implementation process than Waterfall, because the project evolved through iterative testing and refinement.

The chapter then explained why the selected frameworks and libraries are appropriate to the system architecture. FastAPI supports the backend service layer, Streamlit supports the frontend, Gemini supports narrative generation, yfinance and feedparser support evidence retrieval, matplotlib supports charts, ReportLab supports PDF creation, Docker supports deployment, and pytest supports validation.

The literature review therefore establishes a clear research gap and a clear technical rationale for the proposed system. The next chapter uses that foundation to define the system analysis in terms of functional requirements, non-functional requirements, feasibility, and architecture.

Chapter 3: System Analysis

3.1 Introduction

System analysis is the stage in which the project is examined from the perspective of what it must do, how it must behave, and whether it is feasible to implement within the available resources. In an MSc Data Science project, this chapter is expected to move beyond broad problem statements and define the actual operational requirements of the system. It should show that the proposed application is not just an idea, but a structured software solution with a clear function, clear boundaries, and a clear technical basis.

For the present project, the system analysis chapter is especially important because the application combines several interacting components: a natural-language frontend, an API backend, asynchronous orchestration, external data retrieval, generative report writing, chart rendering, PDF production, and live progress streaming. The system must therefore be analysed as a pipeline rather than as a single function or a single page application. Each stage has its own role, its own data dependencies, and its own failure conditions. A proper analysis of the system must reflect that reality.

This chapter describes the functional and non-functional requirements, the user requirements, the feasibility of the system, the high-level architecture, the data flow diagrams, and the use case structure. The analysis is grounded in the actual implementation. In particular, the current codebase exposes a FastAPI backend with report streaming and download routes, a Streamlit frontend, a research agent that gathers evidence in parallel, a report agent that writes section-wise narratives, and a job store that tracks request state. These concrete implementation decisions are what make the analysis meaningful.

3.2 Functional Requirements

Functional requirements describe what the system must do from the user's point of view and from the perspective of the application workflow. For this project, the requirements are derived directly from the implemented pipeline and the exposed API contract.

Table 3.1 here: Functional requirements

ID	Functional Requirement	Priority	Implementation Evidence
FR-01	Accept natural-language stock query	High	Streamlit input form + API request
FR-02	Optional explicit ticker input	High	Streamlit optional ticker field
FR-03	Resolve ticker when absent	High	Ticker resolution tool in orchestration flow
FR-04	Collect market and fundamentals data	High	Research agent market/fundamentals tools
FR-05	Collect recent news and SEC filings	High	News and SEC tools
FR-06	Generate charts for report	High	Chart builder service
FR-07	Generate section-wise narrative	High	Report agent + Gemini prompts
FR-08	Assemble downloadable PDF	High	PDF generator + download route
FR-09	Stream live progress updates	High	SSE endpoint and UI timeline
FR-10	Expose job status and download APIs	Medium	Status/download routes

Purpose: This table should list each functional requirement with an identifier, a short description, and the implementation source that supports it.

3.2.1 Functional Requirement Set

The proposed system must support the following functions.

First, the system must accept a natural-language stock analysis request. The user should be able to enter a query such as “Generate a report for Nvidia stock” without needing to know the exact ticker in advance. This requirement is supported by the Streamlit user interface and by the request schema exposed through the API.

Second, the system must support optional explicit ticker input. This is important because some users will know the ticker symbol already and prefer to bypass resolution. The backend must therefore accept both a plain query and an explicit ticker field.

Third, the system must resolve the relevant ticker and company name when the ticker is not explicitly provided. This is handled by Gemini-based ticker resolution. The system should extract a canonical ticker and a readable company name from the user request and should fall back gracefully if extraction fails.

Fourth, the system must gather market data for the selected stock. This includes price history, recent price movement, and technical indicators such as moving averages and returns over common time windows.

Fifth, the system must gather company fundamentals. This includes company profile data, quarterly financials, and valuation metrics where available. These data are needed to support the narrative sections of the final report.

Sixth, the system must gather news headlines and SEC filing information. Financial research is not complete without recent catalysts and regulatory context, so the report needs both news and filings.

Seventh, the system must generate charts from the collected data. The current system generates a price chart, a revenue chart, and a margins chart. These charts are not decorative; they are part of the analytical evidence included in the report.

Eighth, the system must generate section-wise report narratives. The project is not meant to present raw data alone. It must transform that data into readable financial analysis sections, and that is the role of the report agent.

Ninth, the system must assemble the narratives and charts into a PDF file. The final output must be downloadable and professionally formatted because that is the artifact expected by the academic report and by the user workflow.

Tenth, the system must stream progress updates to the user interface. Since report generation can take time, the user must see which stage is active and whether the process is moving forward.

Eleventh, the system must expose a status endpoint and a download endpoint. These routes are necessary to support polling, UI integration, and artifact retrieval.

Twelfth, the system must support containerised deployment. The codebase includes Docker and deployment-related configuration, so the system should be analysed as a deployable application rather than a local script.

Thirteenth, the system must support testing. A functional academic project must be verifiable, and the current test suite provides evidence that the core modules and routes behave as expected.

3.2.2 Requirement Traceability

These functional requirements can be traced to the actual modules in the implementation. The frontend query form is implemented in the Streamlit application. Request validation is represented by the Pydantic request models. Ticker resolution is handled by the Gemini-based ticker utility. Data retrieval is implemented in the research tools and the research agent. Chart production is handled by the chart builder service. Report writing is handled by the report agent. Document generation is handled by the PDF generator. Status tracking is handled by the in-memory job store. API access is handled by the FastAPI router.

This traceability matters because it shows that the functional requirements are not theoretical. They are directly supported by code.

3.3 Non-Functional Requirements

Non-functional requirements describe how the system must behave rather than what specific output it must produce. For a report-generation system, these qualities matter because the usefulness of the output depends on timeliness, reliability, clarity, and maintainability.

Table 3.2 here: Non-functional requirements

ID	Non-Functional Requirement	Target	Current Status
NFR-01	Usability	Simple input-to-report workflow	Achieved
NFR-02	Responsiveness	Visible progress during long tasks	Achieved via SSE
NFR-03	Reliability	Graceful fallback on partial failures	Partially achieved
NFR-04	Maintainability	Modular code structure	Achieved
NFR-05	Testability	Unit + integration tests	Achieved
NFR-06	Portability	Local + containerized execution	Achieved
NFR-07	Security baseline	Env-based secrets and validation	Partially achieved
NFR-08	Scalability	Support future distributed execution	Future scope

Purpose: This table should list each non-functional requirement with a short explanation and the code or deployment feature that satisfies it.

3.3.1 Performance

The system should complete each major stage within a reasonable time. Because the workflow uses external APIs and model calls, it cannot be instant, but it should remain responsive. The use of asynchronous orchestration and parallel research tasks is an important performance measure. Instead of waiting for one data source to finish before starting the next, the system launches multiple research tasks in parallel and then awaits them with tolerance for individual failures. This reduces overall waiting time and improves user experience.

3.3.2 Reliability

The system should continue operating even if one data source fails. In financial research, external services can be slow, incomplete, or temporarily unavailable. A reliable system should therefore avoid collapsing the entire report just because one source has a problem. The research agent is designed with error isolation so that market data, fundamentals,

news, and SEC filings are collected independently. If one source fails, the system records the error and continues with the remaining data where possible.

3.3.3 Usability

The system should be simple enough for a non-technical user to operate. A user should not need to understand APIs or data pipelines to generate a report. This requirement justifies the Streamlit interface, the natural-language input model, and the live progress timeline. The goal is to reduce friction between the user's intent and the final report artifact.

3.3.4 Security

The system should protect secrets, validate input, and avoid unnecessary attack surfaces. API keys should not be hard-coded, and the container should not run as root. Input should be constrained using request validation. Because the project does not use a relational database, SQL injection is not a concern in the current MVP, but standard web security practices still apply.

3.3.5 Maintainability

The system should be modular enough to support later changes. This is why the code separates agents, tools, services, models, schemas, and routers. If the data source changes, the tool module can be updated without rewriting the whole application. If the report format changes, the PDF generator and report agent can be updated independently.

3.3.6 Scalability

The system should be capable of running in a containerised environment and of being extended into a more persistent architecture later. The present MVP uses an in-memory

job store, which is appropriate for a single-process report generator, but the design already leaves room for replacing it with Redis or another shared store.

3.3.7 Portability

The system should run consistently across environments. Docker support contributes to portability by making the application reproducible in development, testing, and deployment environments.

3.4 User Requirements

The system has more than one user profile in practice, even if the immediate interface is simple.

The primary user is the person requesting the stock report. That user needs a clear form, a simple query field, live progress, and a downloadable PDF. They should not need to understand the internal architecture.

The secondary user is the developer or maintainer. That user needs module separation, a clean API, test coverage, and manageable deployment files so that the system can be changed or extended safely.

The tertiary user is the deployer or administrator. That user needs containerised execution, environment variable support, and cloud-oriented configuration. The deployment design in the current codebase reflects this requirement.

3.5 Feasibility Study

Feasibility analysis determines whether the system can be built effectively with the available technology, budget, and operating model.

3.5.1 Technical Feasibility

The system is technically feasible because it uses a stack that is already proven in production-oriented Python development. FastAPI is suitable for asynchronous API handling. Streamlit is suitable for rapid interface development. Gemini can handle the language-generation tasks. yfinance, feedparser, and SEC EDGAR provide accessible external information sources. matplotlib and ReportLab provide the visual and document output layers. The technical stack is therefore practical, current, and compatible with the project goals.

The project is also technically feasible because each major subsystem is modular. The frontend does not depend on the internal implementation details of the research tools. The API layer does not depend on the visual structure of the PDF. The report generation stage depends on the research bundle, not on the UI. This modularity reduces technical coupling and improves build feasibility.

3.5.2 Economic Feasibility

The system is economically feasible because most of the core components are free or low-cost. The project uses free or publicly accessible data sources where possible. The implementation does not require commercial financial terminal access. The report generation pipeline uses standard Python libraries and a model-driven service layer rather than a large enterprise licensing stack. This makes the project realistic for a student budget.

The current design also avoids unnecessary infrastructure cost. The in-memory job store removes the immediate need for a database server. Docker packaging keeps deployment light. The cloud-oriented design can later be extended, but the MVP does not require expensive infrastructure to be demonstrated as a working project.

3.5.3 Operational Feasibility

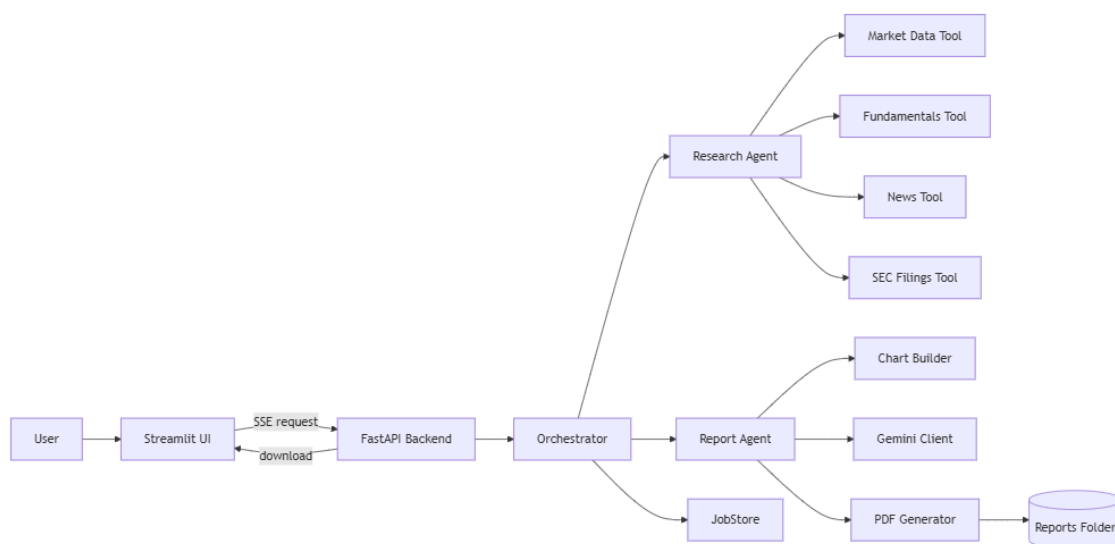
The system is operationally feasible because its workflow matches the way a user would naturally think about the task. A user asks for a report, waits while the system gathers evidence, and receives a polished report artifact. The Streamlit interface makes the workflow easy to operate, and the SSE progress updates make the waiting period understandable rather than opaque.

Operational feasibility is also supported by the application structure. The system can be run locally during development and can be containerised for deployment. The project includes the necessary API routes, environment configuration, and testing evidence to show that the workflow is not only conceptually workable but practically executable.

3.6 System Architecture

The system architecture describes the overall structure of the solution and the relationship among its major components. The current implementation uses a three-layer architecture: the user interface layer, the application/orchestration layer, and the data and output layer.

Figure 3.1



Explanation: This figure should show the Streamlit UI at the top, the FastAPI backend and orchestrator in the middle, and the external data sources plus report generation services beneath. It should make clear that the system flows from user request to evidence gathering to final PDF output.

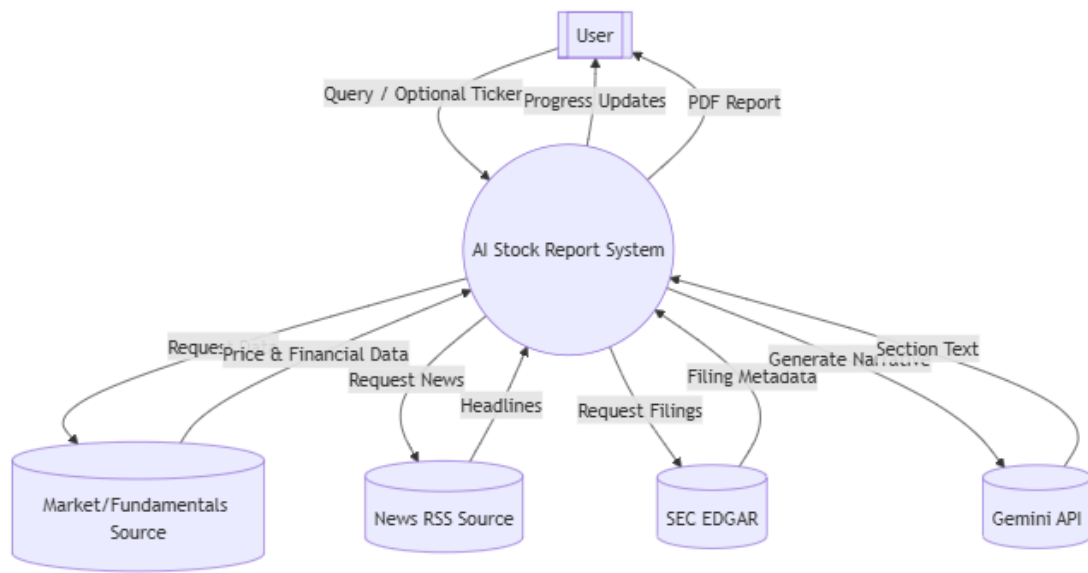
This architecture is appropriate because it separates concerns cleanly. The Streamlit layer focuses on interaction. The FastAPI layer focuses on service exposure. The orchestrator focuses on workflow control. The research agent focuses on evidence acquisition. The report agent focuses on narrative generation. The chart builder and PDF generator focus on presentation. This separation supports maintainability and testing.

3.7 Data Flow Diagrams

Data Flow Diagrams are useful because the system is fundamentally a processing pipeline. They show how data enters the system, how it moves between processes, and where it ends up.

3.7.1 DFD Level 0

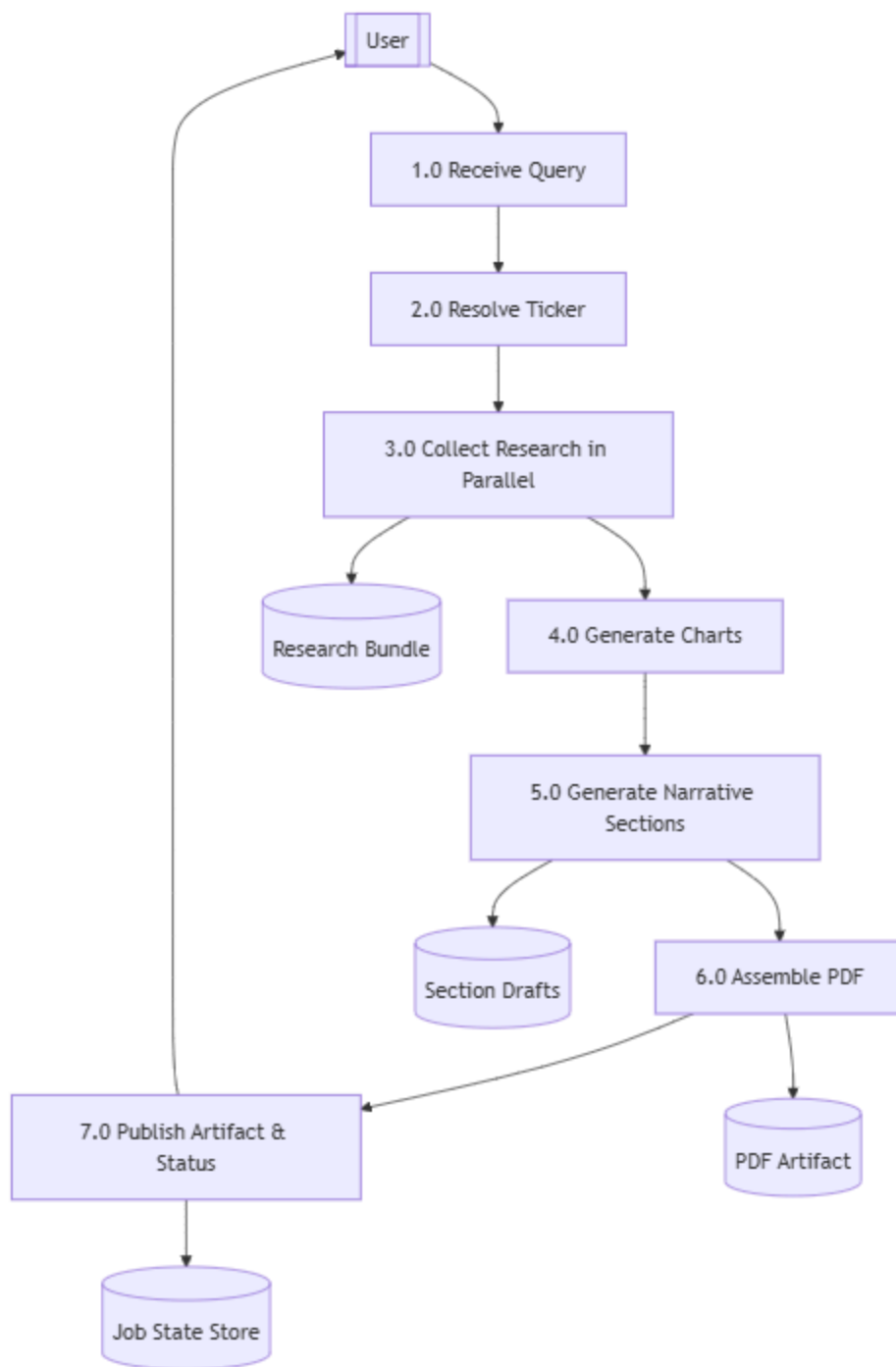
At Level 0, the system can be viewed as a single process that receives a user query and produces a finished PDF report. External entities include the user and the external services used for evidence retrieval and generation.



Explanation: This figure should present the system as one central process with external entities such as the user, Gemini, yfinance, Google News RSS, and SEC EDGAR. It should show the overall input-output relationship without internal decomposition.

3.7.2 DFD Level 1

At Level 1, the system is decomposed into more specific processes: query processing, data aggregation, report generation, and delivery. This decomposition reflects the actual code structure more accurately than the Level 0 view.

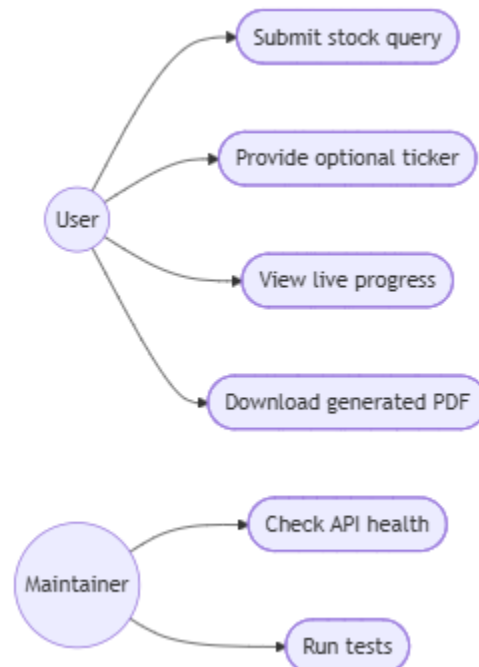


Explanation: This figure should show the internal stages of the pipeline, including the job store, the report generation stage, and the reports directory where PDFs are saved.

The DFD is useful because it makes the data dependencies visible. The report generation stage cannot occur until the research bundle exists, and the download stage cannot occur until the PDF file has been created. The DFD therefore supports understanding of both the normal flow and the dependency order.

3.8 Use Case Diagram

Use case analysis describes how the user interacts with the system from a functional point of view. In this project, the user's main goal is to generate a report, but several sub-actions are involved in achieving that goal.



Explanation: This figure should show the user as the primary actor and the system as the central service. It should include use cases such as submit query, resolve ticker, gather data, generate report, monitor progress, and download PDF.

A useful use case breakdown for this project is as follows:

- Submit stock analysis request
- Optionally provide explicit ticker
- Monitor live progress updates
- Receive finished PDF report
- Poll job status when needed
- Download report artifact

A second set of actors can be shown in the diagram as supporting external services:

- Gemini API for ticker resolution and narrative generation
- yfinance for market and company data
- Google News RSS for recent headlines
- SEC EDGAR for filing metadata

This use case structure is useful because it reflects the actual interaction model of the application. The user does not need to interact directly with the external data sources. Instead, the system performs that work on the user's behalf.

3.9 Chapter Summary

This chapter analysed the proposed system in terms of what it must do, how it must behave, and whether it is feasible. The functional requirements were derived from the application workflow and API design. The non-functional requirements covered performance, reliability, security, usability, maintainability, scalability, and portability. The feasibility study showed that the system is technically, economically, and operationally realistic.

The architecture, DFDs, and use case analysis demonstrated that the system is best understood as a modular pipeline rather than as a single interface or a single model call. That view aligns with the actual implementation and prepares the ground for the next chapter, which will move from analysis into design and structural detail.

Chapter 4: System Design

4.1 Introduction

The purpose of the system design chapter is to translate the analytical findings of the previous chapter into a concrete structural design for the proposed application. If Chapter 3 explained what the system must achieve, Chapter 4 explains how the system is organised to achieve it. In an implementation-focused MSc Data Science report, this distinction is important. The design chapter should not merely repeat the requirements. It should show how the major responsibilities of the system are distributed across modules, interfaces, data structures, and user interaction surfaces.

The AI stock report generator is designed as a modular, pipeline-driven application. It accepts a natural-language request through a web interface, routes the request through an API layer, coordinates multiple research tasks in parallel, generates narrative sections using a language model, constructs charts, and assembles the final report as a PDF artifact. That behaviour cannot be represented effectively as a single block of code. It requires a structured design in which each component has a clear responsibility and a well-defined relationship with the others.

This chapter therefore presents the design from multiple perspectives. First, it examines the user-facing interactions through use case modelling. Second, it explains the object and module structure through a class diagram. Third, it describes runtime coordination through a sequence diagram and an activity diagram. Fourth, it documents the runtime data model through an ER-style representation and conceptual schema discussion. Fifth, it defines the API and interface design. Finally, it explains the user interface states and the deployment-aware component layout. Together, these design views provide a complete picture of how the system is organised.

An important design note must be made at the beginning of this chapter. The current MVP implementation does not use a traditional relational database. Instead, it uses an in-memory job store for active request state and a filesystem-backed reports directory for generated PDF artifacts. Because the university requirement explicitly asks for an ER diagram, database schema, and table structures, these sections are included in conceptual

form. They document the structured runtime data relationships of the system without falsely claiming the existence of a SQL schema that has not been implemented.

4.2 Use Case Design

Use case design describes the system from the perspective of actors and interactions. It is useful in this project because the application appears simple from the user's point of view, but actually performs several internal stages to satisfy that request. A user does not interact directly with market data services, filing APIs, or the language model. Instead, the system abstracts those operations behind one workflow.

The primary actor in the system is the end user. This actor initiates report generation by entering a natural-language stock request in the Streamlit interface. The user may also optionally provide an explicit ticker symbol. Once the request is submitted, the system performs a chain of internal use cases: ticker resolution, data collection, chart generation, narrative generation, PDF assembly, status reporting, and artifact delivery.

Supporting actors include the Gemini API, yfinance-based data retrieval, Google News RSS, and SEC EDGAR. These supporting actors are not end users, but they are essential collaborators in the workflow. The system interacts with them to obtain or generate the information necessary for the report.

The main use cases that should appear in the diagram are as follows:

- Submit stock analysis request
- Provide optional ticker symbol
- Resolve ticker and company name
- Gather market data
- Gather company fundamentals
- Gather recent news
- Gather SEC filings
- Generate analytical charts

- Generate report sections
- Assemble PDF report
- Monitor live progress updates
- Poll job status
- Download report artifact

This use case structure makes the system easier to explain because it presents the entire flow in terms of user intent and service response. It also prepares the reader for the more structural views in the rest of the chapter.

4.3 Class Design

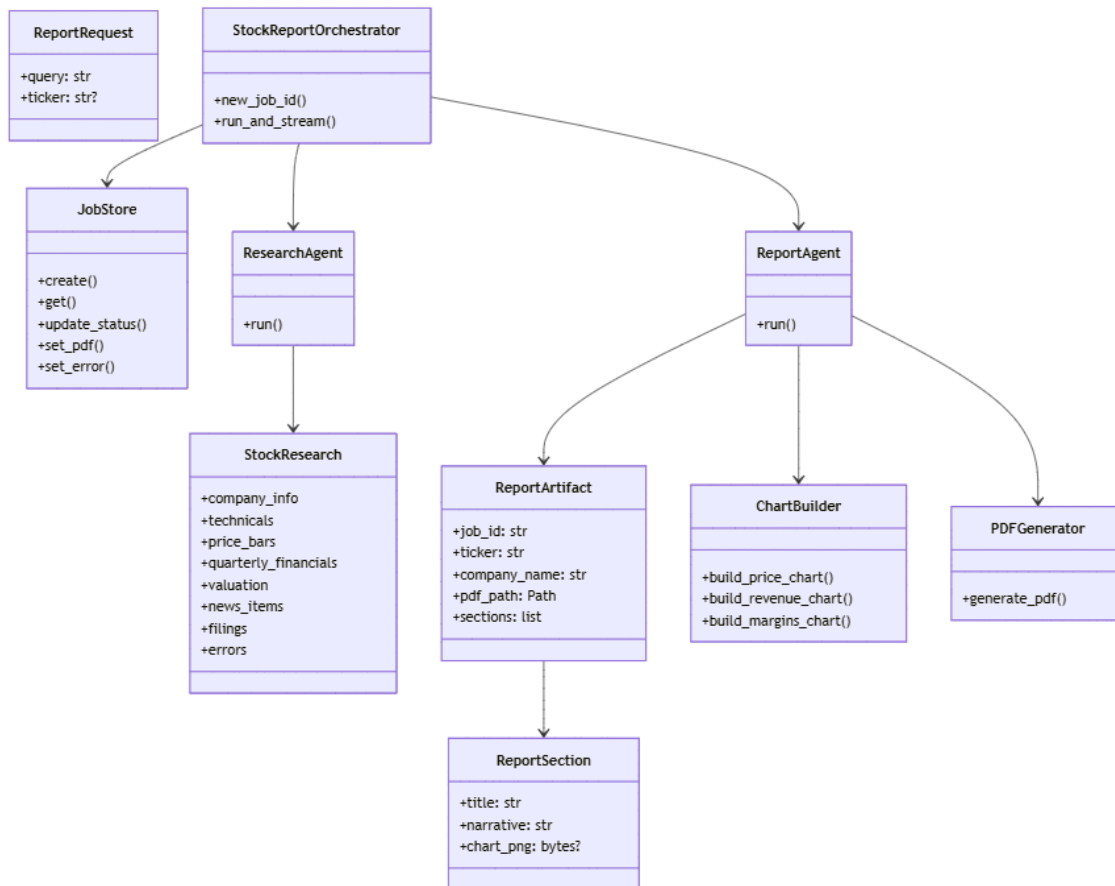
The class design represents the static structure of the system. In this project, the class view is especially useful because the application is intentionally modular. The orchestrator, agents, models, services, and routes are separated to improve maintainability, testability, and clarity.

At the centre of the design is the `'StockReportOrchestrator'`. This class is the main coordination point for the report pipeline. It holds references to the Gemini client, the job store, the research agent, and the report agent. Its main responsibility is to accept a request context and drive the full process from ticker resolution to PDF completion.

The `'ResearchAgent'` is responsible for evidence collection. It does not manage user interaction or output rendering. Instead, it gathers data from multiple sources and packages the result into a `'StockResearch'` object. The `'ReportAgent'` is responsible for converting that research bundle into section narratives and charted report sections, and for triggering the final PDF generation step.

The `'JobStore'` is the runtime state manager. It keeps track of active and completed jobs using `'JobRecord'` objects. Even though it is currently implemented in memory, it plays the same conceptual role that a persistence layer would play in a larger distributed system.

The model layer provides strongly structured domain objects. `StockResearch` acts as the central aggregate for collected evidence. It is composed of `CompanyInfo`, `TechnicalSnapshot`, `PriceBar`, `QuarterlyFinancials`, `ValuationMetrics`, `NewsItem`, and `FilingSummary`. The report layer uses `ReportArtifact` and `ReportSection` to represent the final output before it is rendered to disk.



Explanation: This figure should show the relationships among `StockReportOrchestrator`, `ResearchAgent`, `ReportAgent`, `GeminiClient`, `JobStore`, `JobRecord`, `StockResearch`, and the related domain models such as `CompanyInfo`, `TechnicalSnapshot`, `QuarterlyFinancials`, `ReportArtifact`, and `ReportSection`.

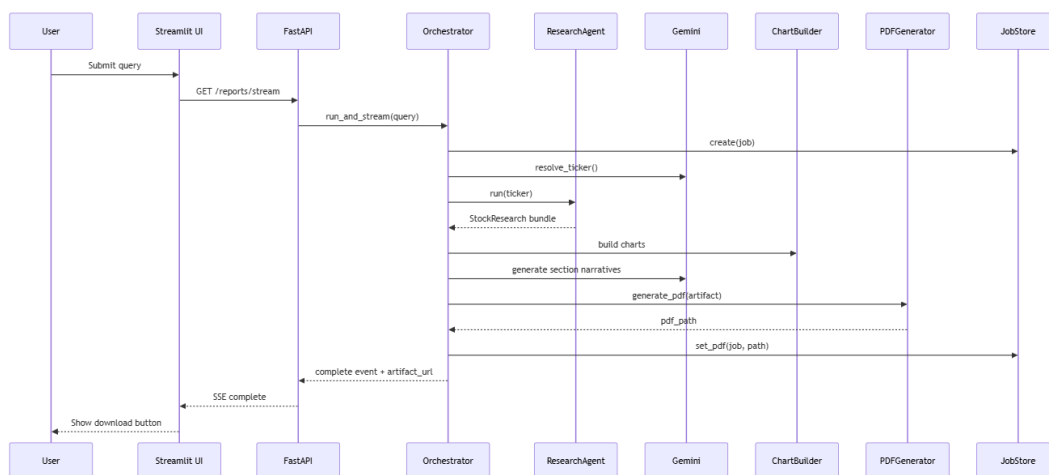
The class design shows that the application follows a composition-heavy, service-oriented structure rather than deep inheritance. The only meaningful abstraction boundary in the agent layer is the base agent pattern. Most of the design is built around

clear responsibilities and domain-specific data structures. This is a practical design choice because it keeps the code understandable and supports future extension.

4.4 Sequence Design

The sequence diagram describes the dynamic interaction among the system components for a normal report-generation request. This view is critical because the application is pipeline-driven. The order of operations is part of the correctness of the design. A PDF cannot be produced before research is complete, research cannot proceed before ticker resolution, and the frontend cannot provide a download action before the report artifact exists.

In the current design, the user begins the process from the Streamlit UI. The frontend opens a streaming request to the backend. The backend route creates a job identifier and starts the orchestrated pipeline. The orchestrator resolves the ticker, creates a job record, launches the research agent, receives the completed research bundle, calls the report agent, receives the final PDF path, updates the job state, and emits a completion event. The frontend listens to the streamed events throughout the process and, once complete, uses the download endpoint to fetch the generated report.



Explanation: This figure should show the chronological interaction among User, Streamlit UI, FastAPI route, `StockReportOrchestrator`, `GeminiClient`, `ResearchAgent`, external data sources, `ReportAgent`, `ChartBuilder`, `PDFGenerator`, and `JobStore`.

The sequence model is useful because it makes it clear that the system is neither fully synchronous nor fully event-driven in the distributed-system sense. Instead, it is an asynchronous application pipeline with a streaming frontend interface. That combination is one of the defining aspects of the design.

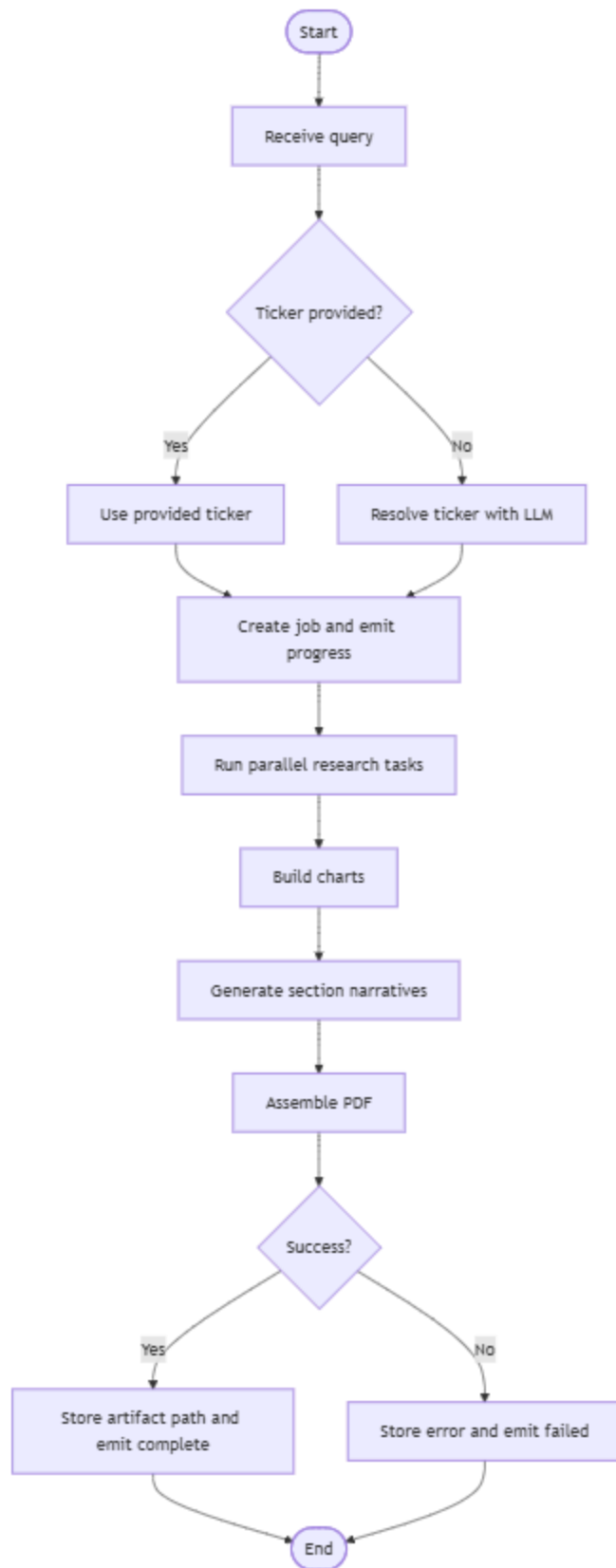
4.5 Activity Design

Where the sequence diagram shows interaction order among objects, the activity diagram shows control flow and branching logic. This is useful for the present project because the workflow contains both decisions and parallel actions.

A key decision appears at the start of the pipeline: whether an explicit ticker was provided by the user. If it was, the system can skip LLM-based ticker resolution. If it was not, the system must call the ticker resolution tool. After that, the system proceeds into the research stage.

The research stage itself contains the most important concurrency point in the application. Market data, fundamentals, news, and SEC filing collection are launched in parallel. These tasks are then awaited independently, and failures are isolated so that the entire report does not necessarily fail because one source is unavailable.

After research, the pipeline proceeds to chart generation, section writing, PDF assembly, status update, and completion event emission. At several stages, progress messages are emitted to the frontend.



Explanation: This figure should show the complete workflow from receiving the query to completing the report, including the branch for optional explicit ticker input and the parallel branch for multi-source research tasks.

The activity view is especially valuable for discussing fault tolerance. Unlike a rigid linear process, the design explicitly allows partial evidence gathering. This makes the system more resilient and supports better user experience.

4.6 ER Diagram and Runtime Data Model

Although the current implementation does not use a relational database, the system still maintains structured runtime entities and relationships that can be documented using an ER-style model. This satisfies the academic need to represent how data is organised while remaining honest about the actual architecture.

The central runtime entity is ``JobRecord``. It represents a report request from creation to completion or failure. A ``JobRecord`` contains the job identifier, query, ticker, company name, status, output PDF path, error information, and recorded progress events.

A second central entity is ``StockResearch``. This entity represents the full evidence bundle collected by the research agent. It is not stored in a relational database in the current MVP, but conceptually it aggregates multiple related entities: company information, technical snapshot, price bars, quarterly financials, valuation metrics, news items, and filing summaries.

A third major entity is ``ReportArtifact``, which represents the report before and during PDF rendering. It contains the output path and the list of ``ReportSection`` objects. Each ``ReportSection`` has a title, a narrative body, and optionally a chart image.

Explanation: This figure should show `JobRecord`, `StockResearch`, `ReportArtifact`, and `ReportSection` as the primary conceptual entities. `StockResearch` should be shown as aggregating `CompanyInfo`, `TechnicalSnapshot`, `PriceBar`, `QuarterlyFinancials`, `ValuationMetrics`, `NewsItem`, and `FilingSummary`.

The reason for presenting this as a runtime data model is that it reflects the true architecture. The application is stateful during execution, but that state is kept in memory and in generated files rather than in database tables. This approach is acceptable for an MVP and should be explained clearly in the final report.

4.7 Database Schema and Table Structures

The university requirement explicitly asks for database schema and table structures. Since no relational database is currently implemented, the most accurate way to satisfy this requirement is to document a conceptual schema that mirrors the structured runtime data.

Table 4.2 here: Runtime entity and structure design

Entity	Core Fields	Purpose
ReportRequest	query, ticker?	Input contract
JobRecord	job_id, query, ticker, status, pdf_path, error	Runtime state tracking
StockResearch	company info, technicals, financials, news, filings, errors	Evidence bundle
ReportSection	title, narrative, chart_png?	Section output unit
ReportArtifact	metadata + sections + pdf path	Final report data model

Purpose: This table should define the logical structure of the key runtime entities, including field names, data types, and roles.

The following conceptual structures should be included.

4.7.1 JobRecord Structure

The `JobRecord` structure should include:

- `job\_id` : string
- `query` : string
- `ticker` : string
- `company\_name` : optional string
- `status` : enum-like string (`pending`, `running`, `done`, `failed`)
- `pdf\_path` : optional file path
- `error` : optional string
- `progress\_events` : list of event dictionaries

4.7.2 StockResearch Structure

The `StockResearch` structure should include:

- `ticker`
- `company`
- `technicals`
- `price\_history`
- `quarterly\_financials`
- `valuation`
- `news`
- `filings`
- `errors`

4.7.3 ReportArtifact Structure

The `ReportArtifact` structure should include:

- `job\_id`

- `ticker`
- `company\_name`
- `pdf\_path`
- `sections`

4.7.4 ReportSection Structure

The `ReportSection` structure should include:

- `title`
- `narrative`
- `chart\_png`

This section should also explicitly note that a future production system could map these conceptual entities into a persistent database such as Redis, DynamoDB, or a relational schema if multi-user support and long-term storage were required.

4.8 API Design

The API design of the system is straightforward but important because the application is built around a service boundary between the frontend and backend. The API is implemented in FastAPI and exposes a small number of focused endpoints. That is appropriate for the project because the system is task-oriented rather than CRUD-oriented.

Table 4.1 here: API endpoint design

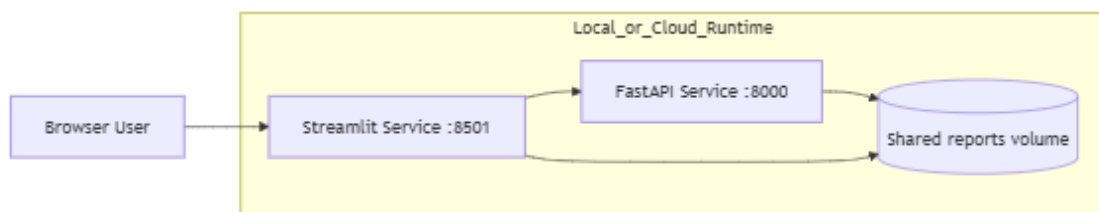
Endpoint	Method	Purpose	Key Response
/health	GET	Service health check	status=ok
/reports	POST	Create asynchronous report job	202 + job_id
/reports/stream	GET	Start and stream report generation	SSE events
/reports/{job_id}/status	GET	Retrieve job status	running/done/failed
/reports/{job_id}/download	GET	Download generated PDF	File response

Purpose: This table should list the method, path, input, output, and purpose of each route in the application.

The main routes are as follows.

First, the `/health` endpoint provides a health check used to verify backend availability. Second, the `POST /reports` endpoint creates a report job and returns a job identifier. Third, the `GET /reports/stream` endpoint opens the SSE stream and drives the main report-generation workflow. Fourth, the `GET /reports/{job\_id}/status` endpoint allows a client to poll for job status. Fifth, the `GET /reports/{job\_id}/download` endpoint serves the generated PDF file if it exists and if the job is complete.

The streaming endpoint is one of the most interesting parts of the API design because it does more than return JSON once. It streams a series of events representing progress through the pipeline. Those event types include `job\_start`, `parse`, `research`, `report`, `complete`, and `error`. This event design allows the frontend to render a live progress timeline rather than waiting for a single final response.



The API design reflects the nature of the application well. It is narrow, explicit, and centred on one business capability: generating and delivering a stock report.

4.9 UI/UX Wireframe Design

The user interface is implemented in Streamlit and is intentionally simple. This is appropriate for the project because the goal is not to build a complex navigation-heavy web product. The goal is to provide one efficient workflow: enter a request, observe the progress, and download the report.

The UI can be understood in three main states.

The first state is the initial input state. The page presents a title, a query field, an optional ticker field, and a generate button. This state should communicate clearly that natural-language input is acceptable.

The second state is the in-progress state. After submission, the interface shows a live progress area where messages are appended as the backend moves through parsing, research, report writing, and completion. This state is important because it reduces uncertainty and gives visibility into long-running work.

The third state is the completed state. Once the report is ready, the interface shows success feedback and a download button for the PDF artifact. This final state is designed to make the output immediately accessible.

Explanation: This figure should depict the three main interface states: blank input form, live progress timeline, and completed download-ready state.

Screenshot placeholders for later insertion:

- Screenshot here: Streamlit blank form
- Screenshot here: Streamlit progress timeline
- Screenshot here: Streamlit completed report screen

The UI design is intentionally minimal, but that is a strength in this project. It reduces cognitive load and keeps attention on the main workflow.

4.10 Deployment and Component Design

Although deployment is discussed in more detail later in the implementation chapter, it is still useful to include a high-level deployment or component design in Chapter 4 because it clarifies how the application parts relate structurally.

The design contains two main runtime services: the Streamlit UI and the FastAPI backend. These services can run locally via Docker Compose or can be deployed as separate containerised services in a cloud environment. The backend depends on a reports directory for generated artifacts and on environment configuration for model credentials and runtime settings. It also depends on external services such as Gemini, market data sources, Google News RSS, and SEC EDGAR.

Explanation: This figure should show the two-service architecture, the reports storage path, and the external services used by the backend. If a cloud design is drawn, it can also show separate frontend and backend services with shared storage and environment-based secrets.

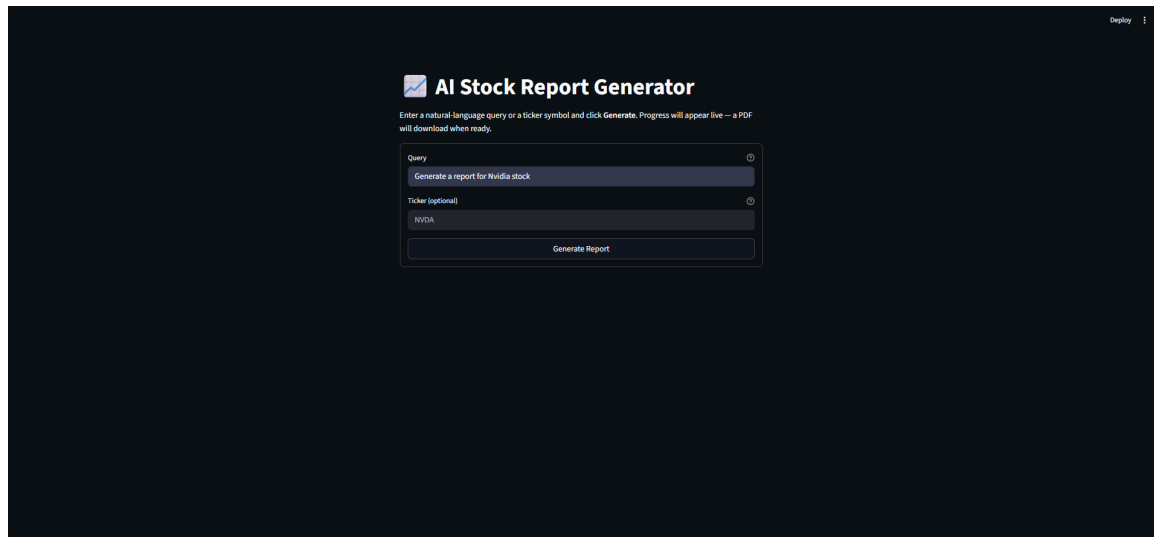
This component-level view is valuable because it makes clear that the application is not a single monolith from the user's perspective. It is a small but well-defined service system.

4.11 Chapter Summary

This chapter transformed the system requirements into an explicit design. The use case model described the application from the perspective of actor interactions. The class diagram identified the core structural elements of the implementation. The sequence and activity diagrams explained the runtime flow, including asynchronous processing and progress streaming. The ER-style model and conceptual schema documented the structured runtime data even in the absence of a relational database. The API and UI design sections showed how the system is exposed to the user, and the deployment view described how the major parts fit together at runtime.

Taken together, these design views show that the AI stock report generator is a modular, maintainable, and extensible application rather than a single script. The next chapter will move from design into implementation, explaining how each of these design elements was realised in the actual codebase.

Chapter 5: System Implementation



5.1 Introduction

This chapter presents the practical implementation of the AI Stock Report system. While the previous chapter focused on system design and architectural planning, the purpose of this chapter is to explain how that design was translated into an executable software solution using real modules, services, user interfaces, and deployment artifacts. The implementation emphasizes modularity, asynchronous processing, reproducible execution, and generation of a usable output artifact in the form of a downloadable PDF stock report.

The developed system is not a single-script prototype. Instead, it is structured as a multi-component application composed of a FastAPI backend, a Streamlit frontend, an orchestration layer that coordinates report generation, specialized agents for research and report writing, chart and PDF generation services, and containerization files for local and cloud deployment. This implementation structure improves maintainability because each part of the system has a clear responsibility. It also improves extensibility because future enhancements, such as persistent storage, stronger access control, or richer analytics, can be added with limited disruption to the existing codebase.

From an operational perspective, the implementation accepts a natural-language request such as asking for a report on a specific stock, resolves the company ticker when necessary, gathers market and company evidence, produces narrative sections with the

aid of a large language model, generates supporting charts, and assembles the final report into PDF format. During this process, progress events are emitted continuously so that the frontend can display a live status timeline rather than leaving the user waiting without feedback. This implementation decision is important because report generation involves multiple dependent processing stages and external data retrieval steps.

The chapter is organized to reflect the actual implementation priorities of the system. It first describes the development environment and execution requirements, then explains the tools and technologies that enabled the build. After that, it presents the module-wise implementation and the main processing logic that drives the report pipeline. Security-related implementation decisions and deployment automation are then discussed. The chapter concludes with screenshot placeholders and a summary of the implementation outcome.

5.2 Development Environment

The development environment for the AI Stock Report system was defined to support modern Python application development, agent-based orchestration, interactive frontend rendering, and containerized deployment. At the foundation of the environment is Python 3.12, which is explicitly declared in the project configuration. This version choice gives the implementation access to current language features, improved typing support, and compatibility with the libraries used in the project.

Dependency and environment management are handled through the uv package manager. This is a practical implementation choice because it simplifies dependency synchronization, project installation, and repeatable environment setup. Rather than relying on loosely managed manual package installation, the project defines its dependencies centrally in the configuration file and uses lock-based synchronization to ensure consistent builds across development and deployment contexts. This contributes directly to implementation reliability by reducing the risk of “it works on one machine but not another” type failures.

The application is organized into two major runtime surfaces. The first is the FastAPI backend, which exposes the report generation endpoints, health endpoint, streaming interface, and download functionality. The second is the Streamlit frontend, which

provides an accessible user-facing interface for entering a query, optionally supplying a ticker, viewing live processing progress, and downloading the completed report. This separation of concerns is important from an implementation standpoint because it allows the backend to focus on API and processing logic while the frontend concentrates on user interaction and display flow.

Configuration values are loaded through a strongly typed settings layer built with pydantic-settings. The settings object includes fields for the Gemini API key, model name, report output directory, API host and port, and research limits such as the number of news items and SEC filings to fetch. Because these settings are environment-driven rather than hard-coded, the same codebase can operate in local development, containerized execution, and cloud deployment with only environment-level changes. This makes the environment portable and reduces the need for source code modification between runtime contexts.

The local development environment can be used in at least two practical ways. The first is a direct Python-based execution workflow in which the backend is started through the project's script entry point and the Streamlit app is launched separately. The second is a Docker-based workflow in which both services run in containers with clearly defined ports and a shared reports volume. The presence of both execution patterns is valuable because it supports fast iteration during development while also making the system easier to demonstrate and package consistently.

A further implementation strength is the deliberate preparation for runtime parity. The same project structure, dependency model, and environment variable strategy are reflected in the Docker files and Compose configuration. As a result, the development environment is not isolated from deployment planning; rather, it is aligned with how the application is expected to run in a more production-like container setting.

5.3 Hardware and Software Requirements

The AI Stock Report system does not require highly specialized hardware, but it does rely on a modern software environment and stable network access because it interacts with external data sources and a cloud-based generative AI service. For academic reporting purposes, it is appropriate to state indicative hardware and software

requirements while also making it clear that the exact machine details can be updated before final submission to reflect the student's actual development device.

Table 5.1 here: Indicative hardware and software requirements

Category	Minimum	Recommended
CPU	Dual-core modern processor	Quad-core or better
RAM	8 GB	16 GB
Storage	10 GB free	20+ GB free
OS	Windows/Linux/macOS	Windows/Linux/macOS
Python	3.12+	3.12+
Internet	Required	Required
Browser	Modern browser	Modern browser
Docker	Optional	Recommended

From a hardware perspective, a standard modern laptop or desktop computer is sufficient for development and local execution. A multi-core processor, at least 8 GB of RAM, and adequate storage for the Python environment, Docker images, generated reports, and development tools are recommended. For smoother multitasking, especially when running both backend and frontend services together or when building Docker images, 16 GB of RAM is preferable. Since the system generates charts and PDF files rather than training machine learning models locally, there is no requirement for a dedicated GPU.

In terms of software, the core requirement is Python 3.12 or later. The project also depends on uv for dependency and environment management. If the containerized workflow is used, Docker and Docker Compose must be available. For the user interface, any modern web browser can access the Streamlit frontend. A code editor or integrated development environment such as Visual Studio Code is suitable for implementation and maintenance activities. Since the application fetches market data, news, and filings from

external services and relies on Gemini for narrative generation, internet connectivity is a mandatory operational requirement rather than an optional convenience.

At the backend level, the FastAPI application requires a valid execution environment in which dependencies such as Pydantic, ReportLab, Matplotlib, Pillow, yfinance, feedparser, and the Google generative AI client are installed correctly. On the frontend side, Streamlit must be able to connect to the backend through a configurable API base URL. This dependency is especially important in containerized operation, where the API is not accessed through localhost from inside the Streamlit container but through the backend service name on the Docker network.

For deployment-oriented execution, the application is designed to run inside Linux-based containers. The existing build and deployment files also show readiness for AWS ECS operation. This means that although the system can be developed on Windows, Linux, or macOS, the deployment target is standardized through containers rather than depending on the characteristics of a particular student machine.

Note: before final submission of the project report, the student should replace the indicative hardware description in this section with the actual CPU, RAM, storage, and operating system details of the computer used for development.

5.4 Tools and Technologies Used

The implementation of the AI Stock Report system relies on a deliberately selected set of tools and technologies, each chosen to solve a specific implementation problem. The stack is practical rather than experimental. It combines a modern Python web framework, a lightweight frontend framework, external data retrieval libraries, a generative AI client, and document generation utilities into a coherent application pipeline.

Table 5.2 here: Tools and technologies used

Component	Technology
Backend Framework	FastAPI
Frontend	Streamlit
Language Model	Google Gemini API
Market/Fundamentals Data	yfinance
News Source	RSS via feedparser
Regulatory Filings	SEC EDGAR API
Charts	Matplotlib (Agg backend)
PDF Engine	ReportLab
Image Handling	Pillow
Validation/Settings	Pydantic + pydantic-settings
Testing	pytest, pytest-asyncio, pytest-mock, pytest-cov
Packaging/Runtime	Docker, Docker Compose
CI/CD/Deployment	GitHub Actions, AWS ECS

Python serves as the primary implementation language because it offers mature support for web APIs, data handling, asynchronous programming, AI integration, and document generation. The project explicitly targets Python 3.12 or later. This provides a stable modern base for type-annotated application code and asynchronous control flow.

The project uses uv as the package and environment management tool. This is important because the implementation includes many dependencies that must remain version-consistent across development and deployment. Centralized dependency management also supports reproducibility in Docker builds and helps maintain clarity over the application's software stack.

FastAPI is used for the backend implementation. This framework is well suited to the project because it provides clear route declaration, automatic request validation through Pydantic models, asynchronous support, and easy generation of JSON responses and streaming responses. In this system, FastAPI handles the report creation endpoint, streaming endpoint, status lookup, download route, and health check. Its asynchronous capabilities align naturally with the event-driven report generation pipeline.

Streamlit is used for the frontend. Rather than building a separate JavaScript-based client, the implementation uses Streamlit to quickly produce a functional and accessible interface for interacting with the backend. This is appropriate for the current project scope because the frontend primarily needs to collect user input, display live progress, and provide a report download action.

The generative AI component is implemented through Google Gemini using the google-genai client library. Gemini is not used as a free-form chatbot in this project. Instead, it is integrated into a structured workflow. It helps resolve a company ticker from a natural-language query when necessary, and it generates the narrative content for report sections based on the evidence bundle compiled by the research stage. This use of AI is therefore task-constrained and embedded within a broader deterministic processing pipeline.

For financial and market information retrieval, the implementation uses yfinance. This library supplies price history, technical data, and fundamentals-related information. For current headlines, the system uses feedparser to process RSS-based news feeds. For regulatory disclosure context, the system includes SEC filing retrieval tools. Together, these sources provide a multi-dimensional evidence base for report generation.

Chart generation is handled with Matplotlib, specifically using the Agg backend. This backend choice is significant because it supports non-interactive server-side image rendering. As a result, the system can generate PNG chart bytes inside backend execution and Docker containers without requiring a desktop display environment. Three charts are produced by the report pipeline: a price history chart, a revenue versus net income chart, and a margins chart.

The final report document is generated with ReportLab, while Pillow is available to support image embedding requirements in the PDF workflow. ReportLab gives the implementation full control over document structure, cover-page styling, chart placement, section rendering, and source listing. This makes it more suitable than a simple plain-text export because the output is intended to resemble a formal report rather than raw text.

Pydantic and pydantic-settings are important supporting technologies. Pydantic is used for schema validation of request and response payloads such as report requests and progress events. Pydantic-settings provides an organized and strongly typed approach to environment-based configuration. Together, they improve reliability and reduce the risk of malformed inputs or inconsistent configuration handling.

Finally, Docker, Docker Compose, GitHub Actions, and AWS ECS provide the operational tooling around the application. Docker packages the API and Streamlit services into separate images. Docker Compose enables both services to run locally with a shared reports volume. GitHub Actions automates image build and deployment steps, while AWS ECS is the intended cloud orchestration target indicated by the deployment workflow. These tools show that implementation was not limited to coding the application itself; it also included practical operational packaging and deployment readiness.

5.5 Module-wise Implementation

The system is implemented as a set of cooperating modules rather than a single monolithic program. This structure improves clarity because each module addresses one implementation responsibility. It also makes the project easier to test, extend, and maintain.

Table 5.3 here: Module responsibilities and implementation roles

Module	Responsibility
app/main.py	API startup and route registration
app/config.py	Environment-driven settings
app/routers/reports.py	Report lifecycle endpoints
app/orchestrator.py	End-to-end pipeline control
app/agents/research.py	Parallel evidence collection
app/agents/report.py	Narrative/charts/PDF orchestration
app/services/chart_builder.py	PNG chart generation
app/services/pdf_generator.py	Final report PDF rendering
app/services/job_store.py	In-memory job state management
streamlit_app.py	User interaction and live progress display

5.5.1 Application Entry Point and Core API Setup

The application entry point initializes the FastAPI backend and registers the report-related routes. It also defines a simple health endpoint used for service checks and container health verification. This is an important implementation detail because both Docker and external monitoring can use the same route to confirm that the API is running correctly. The entry point also includes a development runner that starts Uvicorn using host and port values loaded from configuration.

From an implementation standpoint, the entry point remains intentionally lightweight. It does not contain business logic. Instead, it composes the web application and delegates report handling to the routing and service layers. This is a good engineering choice because it preserves separation between HTTP-level concerns and domain logic.

5.5.2 Configuration and Schema Layer

Configuration is centralized in a typed settings module. This module defines the Gemini API key, Gemini model, reports directory, server binding values, and research-related limits. The use of a typed settings layer gives the implementation two benefits. First, configuration values are documented through code structure and type definitions. Second, environment-driven deployment becomes cleaner because sensitive values such as API keys do not need to be hard-coded into the source.

The schema layer defines the request, progress event, and response models used by the API. This includes validation rules such as the minimum length requirement for the report query. As a result, invalid requests can be rejected consistently before deeper processing begins. The schema layer is therefore a small but important part of implementation quality.

5.5.3 API Routing and Client Interaction Surface

The report routes expose the application's main operational interface. The backend includes endpoints for creating a report job, streaming progress, checking job status, downloading the generated PDF, and checking service health. The existence of both immediate-response and streaming-oriented routes reflects a thoughtful implementation choice. Some operations, such as job creation or status inspection, should return quickly. Others, such as full report generation, are better handled through a streaming mechanism so the user can observe progress.

The streaming route is especially important. It creates a new job identifier, starts the report pipeline, and yields progress events as server-sent events. This allows the frontend to render a live timeline of stages such as parsing, research, report writing, and completion. Instead of forcing the user to poll repeatedly or wait for a silent long-running request, the implementation provides incremental transparency throughout execution.

The download route is also implemented carefully. It validates job existence, checks whether processing is finished, ensures the PDF path exists on disk, and returns the file with a generated filename. The filename is sanitized to avoid unsafe characters. Although this is a relatively small detail, it shows attention to safe operational behavior.

5.5.4 In-Memory Job Tracking

Job state is managed by a dedicated in-memory job store. The store tracks the query, ticker, company name, job status, PDF path, error state, and accumulated progress events for each report request. The implementation uses an `asyncio.Lock` to guard critical mutation operations, which makes the store thread-safe in the context of concurrent asynchronous access.

This choice is appropriate for the current project stage because it keeps the implementation simple and avoids introducing a database prematurely. It also reflects the current scope honestly: the system can track active and completed jobs during process lifetime, but it does not yet provide persistent relational storage across restarts. This is not a defect in the chapter; it is an accurate description of the current implementation boundary.

The job store also supports the operational semantics of the API routes. Status checks, error reporting, and download availability all depend on the information recorded there. In other words, the job store functions as the temporary coordination memory of the application.

5.5.5 Orchestration Layer

The orchestrator is the central control module of the application. Its role is to coordinate the end-to-end report pipeline rather than perform the detailed research or report-writing work itself. This includes resolving the ticker, creating a job record, invoking the research agent, invoking the report agent, updating the job store with the generated PDF path, and emitting progress events to an asynchronous queue that can be streamed to the frontend.

This design is significant because it establishes a single authoritative pipeline entry point. The system does not allow the frontend or route handlers to manually chain together internal services in an ad hoc way. Instead, the orchestrator encapsulates the sequence of

operations and error handling. This makes behavior more predictable and easier to maintain.

The orchestrator also improves user experience. By using an asynchronous queue and emitting progress events at each stage, it supports real-time feedback over server-sent events. If an exception occurs, the orchestrator records the error, updates job state, and emits a terminal error event. This keeps failure handling consistent rather than fragmenting it across multiple modules.

5.5.6 Research Agent

The research agent is responsible for collecting the evidence that will later support the report narrative. It launches four categories of data retrieval work in parallel: market data, fundamentals, news, and SEC filings. This parallel approach is one of the strongest implementation decisions in the project because these activities are independent and would unnecessarily increase latency if performed strictly one after another.

Another important feature of the research agent is fault tolerance. Each retrieval task is awaited separately, and exceptions from one source do not automatically terminate the entire evidence-gathering phase. Instead, failures are logged, error messages are added to an internal list, and sensible fallback values are applied where necessary. This allows the system to continue operating even if one external source becomes unavailable or returns incomplete data. In practical implementation terms, this increases robustness and makes the final system more usable in real network conditions.

When all available data has been collected, the research agent constructs a `StockResearch` bundle containing company information, technical indicators, price history, quarterly financials, valuation data, news items, filings, and any accumulated error messages. This evidence bundle becomes the structured input for the report-writing stage.

5.5.7 Report Agent

The report agent is responsible for transforming research evidence into a structured report artifact. Its implementation includes three main responsibilities: building charts, generating narrative sections through Gemini, and assembling the final PDF through the PDF generation service.

The chart-building phase produces three visual outputs: a price chart, a revenue and net income chart, and a margins chart. These visuals are prepared before the narrative section loop so that relevant sections can include chart images directly in the final PDF artifact.

For the narrative phase, the report agent defines a fixed sequence of ten report sections, including executive summary, company overview, price performance, financial performance, margin analysis, valuation, recent news, SEC filings, risks, and investment thesis. This section list is not hard-coded as one massive text block. Instead, it is expressed as a structured specification with titles, writing instructions, and optional chart attachments. This is a strong implementation pattern because it keeps the report composition transparent and extensible.

The agent also includes logic for cleaning up model output. Since generated text can sometimes contain stray formatting or incomplete endings, the implementation normalizes the narrative and trims visibly cut-off fragments to the last complete sentence when necessary. This is a subtle but important quality-control step because it improves the readability of the final document without requiring manual intervention.

Once all sections are generated, the report agent creates a report artifact object containing metadata, PDF path, and section content, then passes it to the PDF generation service.

5.5.8 Chart Generation Service

The chart builder is a focused service dedicated to producing image data for the final report. It uses Matplotlib configured with the Agg backend, which allows non-interactive rendering in server and container environments. This choice is important because the backend cannot assume the presence of a desktop display.

The service includes dedicated functions for generating the price history chart, the revenue versus net income chart, and the gross versus operating margin chart. Each function transforms structured financial data into a visual format appropriate for report embedding. The generated figures are converted into PNG bytes and returned to the caller rather than being written directly to long-term storage. This is a clean design because it keeps chart generation decoupled from final document assembly.

The implementation also applies consistent styling choices such as background color, figure size, grid appearance, and axis formatting. These decisions support readability and produce visuals that look report-ready rather than merely functional.

5.5.9 PDF Generation Service

The PDF generation service is the final formatting layer of the system. It takes the assembled report artifact and renders it into a formal PDF document using ReportLab. The generated document includes a styled cover page, section headings, narrative paragraphs, embedded chart images, data source listings, and a disclaimer.

A notable implementation detail is the use of a ReportLab Table to render the current cover page with a navy background and centered content. This gives the report a more deliberate presentation than a plain-text export would provide. The service also defines paragraph styles for headings, body text, captions, and sources, which helps maintain formatting consistency across the final document.

Chart images are inserted only when valid PNG bytes are available. Narrative content is split into paragraphs and rendered sequentially. The service creates the reports directory if needed and writes the final PDF to a predictable path based on the job identifier. This path is then stored in the job store for later download.

Because the PDF generator is isolated from the higher-level orchestration and report-writing logic, it can be modified or enhanced in future without redesigning the rest of the application. This reinforces the modular implementation approach.

5.5.10 Streamlit Frontend

The Streamlit interface provides the user-facing surface of the system. It includes a form with a natural-language query field and an optional ticker field. If the user already knows the ticker, it can be supplied directly to bypass the ticker resolution stage. Otherwise, the backend resolves it automatically. This is a useful implementation feature because it supports both expert and non-expert users.

The frontend communicates with the backend through a configurable `API_BASE` environment variable. This is important because the correct backend address differs between local direct execution, Docker Compose networking, and cloud deployment scenarios. The frontend therefore remains implementation-portable rather than being locked to a single host assumption.

Another strong implementation choice is the live progress display. As the backend emits server-sent events, the frontend appends each event to an internal list and re-renders the visible progress timeline. This gives the user transparency into long-running operations such as evidence gathering, section writing, and PDF assembly. When the process completes successfully, the frontend requests the generated PDF and exposes it through a download button.

The frontend is intentionally simple in design, but it is functionally effective. It demonstrates that the system is not only implemented as backend logic but is also usable through an interactive application layer.

5.6 Key Algorithms and Processing Logic

The implementation of the AI Stock Report system is defined not only by its modules, but also by the processing logic that links those modules into a working pipeline. Several core algorithmic and control-flow decisions shape how the system behaves in practice.

The first major step is request interpretation and ticker resolution. When a user initiates report generation, the system receives either a natural-language query alone or a query plus an explicit ticker. If a ticker is supplied, it is normalized and used directly. If not, the orchestrator calls the ticker resolution tool through the Gemini-supported flow to identify the appropriate company symbol. This design balances usability with efficiency. Users who know the ticker can skip a step, while users who do not know it can still interact in natural language.

Once the ticker and company identity are known, the system creates a job record and enters the research phase. The key algorithmic decision here is parallelism. The research agent starts the market data, fundamentals, news, and SEC filing retrieval tasks concurrently. This reduces waiting time because the tasks depend on different external sources and do not need to block one another. The implementation does not treat research as an all-or-nothing stage. Instead, each task is awaited separately and wrapped with its own exception handling path. If one source fails, the agent records the problem and continues with the remaining evidence. This partial-failure tolerance is a practical algorithmic improvement over a brittle serial pipeline.

After evidence has been assembled, the report phase begins. The report agent first builds charts from the collected quantitative data. This ensures that visual material is ready before narrative composition is finalized. The algorithm then serially iterates through a predefined list of report sections. For each section, it constructs a prompt that combines section-specific instructions with the compact research context bundle. This makes the text generation process structured rather than arbitrary. In effect, the system uses generative AI inside a controlled template-driven reporting workflow.

A further quality-oriented algorithm is narrative finalization. Since language model output may sometimes end abruptly or include formatting artifacts, the implementation normalizes the text, removes accidental markdown fences, compresses excessive blank

lines, and trims incomplete endings back to the last complete sentence when needed. This algorithm is small, but it directly improves the perceived quality of generated content and reduces the chance of visibly broken paragraphs in the PDF output.

The system also uses an event-driven progress tracking mechanism. The orchestrator pushes `ProgressEvent` objects into an asynchronous queue as each major stage begins or ends. The streaming endpoint then converts these events into server-sent event messages. The frontend reads the event stream and updates the visual progress timeline in near real time. This is an implementation of asynchronous producer-consumer coordination, where the backend pipeline acts as the producer and the streaming client acts as the consumer. The result is a more responsive and transparent user experience.

Chart generation itself follows a server-side rendering algorithm. Structured financial data is transformed into plotted figures, which are then written into in-memory buffers as PNG bytes. By returning bytes instead of writing separate image files for intermediate steps, the service reduces filesystem complexity and keeps chart generation tightly integrated with the final document creation process.

The last major processing step is PDF assembly. The report artifact, consisting of metadata, section narratives, and optional chart bytes, is passed into the PDF generation service. The service iterates through sections, formats paragraph content, inserts charts where available, appends sources, and writes the final document to disk. The completed PDF path is then stored in the job store, and the orchestrator emits a completion event containing the download URL. From a systems perspective, this closes the loop from initial user request to retrievable artifact.

5.7 Security Features

The current implementation includes several practical security-oriented decisions, even though the project does not yet implement a full enterprise security model. The first important measure is environment-based secret handling. Sensitive configuration values, especially the Gemini API key, are not embedded directly into the source code. Instead, they are loaded through the settings layer from environment variables. This reduces the risk of accidental credential exposure in the repository and makes it easier to manage different environments.

A second important feature is input validation. The FastAPI backend relies on Pydantic schemas to validate request and response structures. For example, the report request requires a non-empty query, and the progress event and response objects follow explicit typed models. This improves robustness and helps prevent malformed inputs from propagating deeper into the system.

The implementation also includes filename sanitization for downloadable report files. Unsafe characters are removed from generated filename components before the file is returned to the client. Although simple, this is an example of defensive implementation practice and reduces the chance of problematic filenames.

At the container level, both runtime Docker images are configured to run as a non-root user named appuser. This is an important operational safeguard because it limits the privileges available inside the container. Running services as non-root is a recognized best practice for reducing the impact of container compromise or accidental misuse.

In the deployment workflow, GitHub Actions is configured to use OpenID Connect for AWS authentication. This is a significant security decision because it avoids reliance on long-lived static cloud access keys stored in the CI/CD environment. Instead, the workflow assumes a role dynamically, reducing credential management risk and aligning with better modern cloud deployment practices.

It is also important to describe the implementation honestly. The current codebase does not include a relational database, and therefore there are no database credentials, SQL access controls, or schema-level protections to discuss. Job state is currently stored in memory. Similarly, the visible implementation does not yet include user authentication, role-based authorization, or tenant isolation. This means that while the system contains sound baseline protections in configuration, validation, containerization, and deployment credentials, it should still be regarded as an application suited to controlled usage environments until stronger access-control features are added.

5.8 Version Control and CI/CD

The implementation is supported by a version-controlled project structure and a deployment workflow designed for repeatable release. Source code organization, dependency declarations, Docker build files, and infrastructure-related artifacts indicate a Git-based development process in which application logic and operational configuration are maintained together.

The CI/CD workflow defined for the project performs several deployment-critical tasks. It checks out the repository, authenticates to AWS using OIDC, logs in to Amazon ECR, builds separate container images for the API and Streamlit frontend, pushes those images with both commit-specific and latest tags, and then triggers fresh deployments of the ECS services. Finally, it waits for the services to stabilize and records a short deployment summary. This is a practical and realistic automation pipeline for the scale of the current project.

Table 5.4 here: CI/CD stages and deployment responsibilities

Stage	Action	Output
Source Checkout	Pull repository code	Build-ready workspace
Auth	OIDC-based AWS credential setup	Secure cloud session
Build	Build API and UI images	Container images
Push	Push images to registry	Versioned image artifacts
Deploy	Trigger ECS service updates	New running tasks
Stabilize	Wait/check service health	Deployment confirmation

An important implementation point is that the application is ready for both local and cloud-oriented operation. Docker Compose provides local two-service execution with a shared reports volume, allowing the API and frontend to operate together in a controlled environment. The GitHub Actions and ECS files extend the same packaging approach to deployment-oriented infrastructure. This shows good implementation continuity from development to deployment.

The codebase also includes unit and integration tests for critical implementation slices such as chart building, PDF generation, ticker resolution, narrative finalization, job store behavior, and API endpoints. These tests contribute to implementation quality, although their detailed analysis belongs in Chapter 6 rather than this chapter.

5.9 Screenshots of Application

This section should contain carefully selected screenshots that demonstrate the implemented system in operation. The screenshots should be inserted into the final Word document after capture, but placeholders are used in this draft so that the chapter structure remains complete.

This figure should demonstrate the initial interaction point of the system and show that the user can begin report generation either with a natural-language request or by providing a ticker directly.

This figure should show the real-time event updates that appear as the backend emits progress events through the streaming endpoint.

This figure should confirm that the system completes processing and exposes the finished PDF as a downloadable artifact.

This figure should demonstrate that the backend service is active and that the API layer is operational.

This figure should illustrate the actual output artifact produced by the implementation, including cover styling or section formatting.

This figure should support the discussion of containerized local deployment readiness.

This figure should show that the implementation includes an operational deployment path beyond local execution.

5.10 Chapter Summary

This chapter has explained the implementation of the AI Stock Report system as an integrated software application rather than a purely conceptual design. The system was implemented using a modular architecture in which the FastAPI backend manages API interaction, the orchestrator controls report generation flow, the research and report agents perform evidence collection and report composition, the chart and PDF services generate visual and document outputs, and the Streamlit frontend provides a practical user interface.

The implementation choices made in the project reflect a balance between simplicity and capability. The use of Python 3.12, uv, FastAPI, Streamlit, Pydantic, Matplotlib, ReportLab, and Docker creates a stack that is practical for academic development while still being technically sound. The orchestration logic, parallel data collection, failure

tolerance, progress streaming, and PDF assembly process show that the application is more than a static demonstration; it is a working end-to-end report generation pipeline.

The chapter also showed that implementation quality extends beyond feature coding. Configuration is environment-driven, containers run as a non-root user, deployment automation uses OIDC-based authentication, and the project includes supporting tests for key implementation areas. At the same time, the chapter has remained honest about the current scope: job state is in-memory, no relational database has yet been introduced, and stronger access-control features remain future enhancements rather than current features.

Overall, the implementation successfully translates the proposed system design into a working application capable of generating stock research reports with live progress feedback, chart support, PDF output, and deployment readiness for both local Docker execution and AWS ECS-oriented deployment. The next chapter can therefore move naturally from implementation description to testing, evaluation, and assessment of system performance..

Chapter 6: Testing and Quality Assurance

6.1 Introduction and Testing Strategy

This chapter presents the testing and quality-assurance activities carried out for the AI Stock Report system after implementation. The purpose of the chapter is not simply to state that the software works, but to show how important parts of the application were verified through structured tests, targeted debugging, and controlled validation of system behavior. Because the project combines asynchronous backend processing, external-data integration, AI-assisted content generation, chart rendering, and PDF generation, testing was an essential part of ensuring that the software behaved predictably and produced usable outputs.

The testing approach adopted in this project was layered. At the most granular level, unit tests were used to verify critical logic in individual modules, such as job-state transitions, narrative cleanup, chart rendering, PDF generation, and ticker-resolution fallback behavior. At the next level, integration tests were used to verify API route behavior, request validation, and expected HTTP responses from the FastAPI backend. In addition to automated testing, limited manual system verification was used during development to confirm that the backend, frontend, and report-generation workflow could be exercised together in controlled runs.

The test suite was implemented using the pytest ecosystem. The project configuration includes pytest, pytest-asyncio, pytest-mock, and pytest-cov as testing tools. In the validated run used for this project stage, all 33 tests passed successfully. These tests are composed of 25 unit tests and 8 integration tests. The use of automated tests was especially important because the system depends on several external services, including Gemini and market-data sources. To keep tests repeatable and independent of external credentials or network availability, the test design relies on mocking for external calls. This improves reliability of the testing workflow, although it also means that the test suite does not by itself prove live end-to-end behavior against real external services.

Table 6.1 here: Testing strategy and scope matrix.

Level	Scope	Tools	Evidence
Unit	Isolated module logic	pytest	25 unit tests
Integration	API route behavior	FastAPI TestClient + pytest	8 integration tests
Manual System	Controlled end-to-end usage checks	Manual runs	UI-to-PDF observed flow

From an academic and engineering perspective, this strategy is appropriate for the current state of the system. It verifies the most failure-prone implementation slices in a deterministic way, supports regression testing after bug fixes, and provides evidence that the core logic behaves correctly under both normal and error scenarios. At the same time, the chapter remains honest about what has not yet been automated, especially full live pipeline testing, frontend automation, and performance testing.

6.2 Unit Testing

Unit testing was used to validate the most important self-contained logic units in the application. Rather than attempting to test every line of code indiscriminately, the unit suite focuses on components where incorrect behavior would have a visible effect on user experience, report quality, or application stability. These areas are the in-memory job store, the narrative finalization logic in the report agent, chart generation, PDF generation, and ticker resolution.

Module/Test File	Focus	Test IDs
test_job_store.py	Job lifecycle and state updates	TC-U-01 to TC-U-08
test_narrative_finalize.py	Narrative cleanup and markup conversion	TC-U-09 to TC-U-14
test_chart_builder.py	PNG chart output validation	TC-U-15 to TC-U-17
test_pdf_generator.py	PDF creation robustness	TC-U-18 to TC-U-21
test_ticker_resolve.py	Ticker parsing and fallback logic	TC-U-22 to TC-U-25

6.2.1 Job Store Testing

The JobStore is the in-memory coordination layer that tracks report jobs during execution. Since job state is used by status endpoints, progress handling, and download availability checks, correctness in this module is important for overall application behavior. The unit tests for JobStore verify that a new record is created with the expected pending state, that existing records can be retrieved correctly, and that unknown identifiers return a null-like result rather than causing a crash. Additional tests verify that status transitions work correctly, that setting a PDF path marks the job as complete, that error recording marks the job as failed, that company-name metadata is stored correctly, and that duplicate job identifiers overwrite previous values in a predictable way.

These tests are important because the current system does not yet use a persistent database. The in-memory store therefore acts as the central temporary record of active and completed work. If its behavior were inconsistent, users could encounter invalid job states, missing download links, or incorrect error reporting. The unit tests help ensure that these risks are controlled within the current architecture.

Representative examples include TC-U-01, which verifies that job creation returns a record with pending status, and TC-U-05, which verifies that storing a generated PDF path automatically changes status to done. Together, these tests show that the job-state lifecycle behaves consistently from creation to completion.

6.2.2 Narrative Finalization Testing

A second important set of unit tests focuses on narrative cleanup and markup conversion in the report-writing path. The report agent uses AI-generated text, and this introduces a special class of quality risks. Language-model output can contain incomplete endings, markdown fences, inconsistent whitespace, or formatting that does not map directly into ReportLab-compatible markup. To reduce these risks, the implementation includes helper logic that normalizes narrative output before it is written into the PDF.

The tests for this area verify that a complete sentence passes through correctly, that text cut off in the middle of a sentence is trimmed back to the last complete terminal character, that question-mark endings are preserved, that markdown fences are removed safely, that an empty string produces a safe fallback instead of causing failure, and that markdown bold notation is converted into ReportLab-safe bold tags. These tests are

particularly important because poor narrative cleanup would reduce the quality of the final report even if the rest of the system were working correctly.

TC-U-10 is a strong example because it verifies the logic used to remove visibly incomplete sentence endings. This was a real quality concern during development, and the test now acts as a regression guard. TC-U-14 is also important because it verifies that bold markdown is transformed correctly rather than being left as raw markdown or converted into broken PDF markup.

6.2.3 Chart Generation Testing

The chart-building service produces the visuals used in the stock report. Since these charts are generated on the backend and later embedded into the PDF, it is necessary to verify that the chart functions return valid image data. The chart-related unit tests therefore check that the price chart, revenue chart, and margins chart all return non-empty byte sequences beginning with the PNG file signature.

This approach is appropriate because the most important correctness property at unit level is that the chart-generation functions return usable PNG output for valid structured financial data. The tests do not attempt pixel-by-pixel image comparison, which would be unnecessarily fragile for this project. Instead, they verify the existence and format of the output artifact, which is what the rest of the system depends on.

TC-U-15, TC-U-16, and TC-U-17 collectively confirm that chart rendering works across the three implemented chart types. This is important because chart failures would directly affect both report completeness and PDF generation flow.

6.2.4 PDF Generation Testing

The PDF generator is the final document-assembly component of the application. It converts the report artifact into a styled PDF containing headings, narrative content, charts, and supporting sections. Since this service produces the final deliverable consumed by the user, testing it is essential.

The PDF unit tests verify that the generator creates a non-empty file on disk, that it still succeeds when a section contains no chart image, that bold markdown inside narrative text does not break PDF generation, and that plain text remains unchanged when markup conversion is unnecessary. These tests address both artifact creation and formatting robustness.

TC-U-18 is particularly significant because it verifies the most basic promise of the service: a report artifact should result in a physical PDF file. TC-U-19 and TC-U-20 extend this by ensuring that optional chart absence or inline markdown formatting does not break report creation. This is important in practice because not every section needs a visual and generated narrative may include emphasis markers.

6.2.5 Ticker Resolution Testing

Ticker resolution is another critical slice because it sits at the beginning of the report-generation workflow. If the ticker cannot be resolved or the resolution logic fails unexpectedly, the entire pipeline is affected. The unit tests for ticker resolution therefore focus not only on the normal success path, but also on malformed or failing AI responses.

These tests verify that valid JSON from the Gemini-backed resolution flow is parsed correctly, that invalid JSON falls back gracefully, that a thrown exception from the AI client does not propagate uncaught, and that an empty ticker returned in JSON also triggers fallback behavior. This is a strong example of defensive testing because it ensures that the system remains usable even when an external AI dependency behaves unexpectedly.

TC-U-24 is especially important because it verifies the fallback path when the external resolution call raises an exception. This test reflects a real bug discovered during testing and later fixed in the implementation, making it both a correctness test and a regression test.

6.3 Integration Testing

While unit tests validate isolated logic, integration tests were used to verify the interaction surface of the FastAPI backend. The goal of these tests is to confirm that the main HTTP endpoints respond with appropriate status codes, accept valid inputs, reject invalid inputs, and behave consistently under normal request patterns.

The integration suite uses FastAPI TestClient together with dependency overrides so that the orchestrator can be mocked. This design choice is important because it allows API behavior to be tested without triggering the full report-generation pipeline or requiring live external dependencies. In other words, the integration tests verify how the API layer behaves as an interface, while still remaining deterministic and fast.

Table 6.3 here: Integration test summary.

Test Area	Expected Behavior	Outcome
Health endpoint	200 + status ok	Pass
Unknown status/download	404 responses	Pass
Report creation valid input	202 + job_id	Pass
Validation for empty query	422 error	Pass
Optional ticker handling	Accepted request	Pass
Health idempotency	Consistent responses	Pass

The available integration tests cover several important cases. TC-I-01 verifies that the health endpoint returns HTTP 200 with the expected payload, which is important for service monitoring and container health checks. TC-I-02 and TC-I-03 verify that unknown job identifiers return 404 responses for status and download endpoints. TC-I-04 verifies that a valid report-creation request returns HTTP 202 and includes a job identifier, which reflects the asynchronous nature of report generation. TC-I-05 verifies input validation by ensuring that an empty query is rejected with a 422 response. TC-I-06

confirms that the optional ticker field is accepted correctly. TC-I-07 verifies that repeated health checks remain consistent, and TC-I-08 checks that the status endpoint returns a valid response shape after report creation, acknowledging that a mocked orchestrator may leave the status unresolved in the in-memory store.

These tests are meaningful because they verify the externally visible contract of the API. Even when internal implementation changes, correct route behavior must remain stable for the frontend and other clients. The integration tests therefore provide an important safeguard against breaking the backend interface while refactoring internal logic.

6.4 System Testing and Manual Verification

The university guideline expects Chapter 6 to discuss system testing in addition to unit and integration testing. In the current project, system-level verification was performed in a limited manual form rather than through a fully automated end-to-end suite. This distinction is important and should be stated explicitly.

During development, the integrated application was exercised through controlled runs in which the FastAPI backend and Streamlit frontend were launched together, user queries were submitted, progress events were observed, and completed reports were downloaded for inspection. These manual checks were useful for confirming that the major system components could operate together as a usable workflow rather than only as isolated modules. In particular, they helped validate the basic end-to-end path from query submission to PDF generation and download.

At the same time, the current project does not include an automated system test that runs the full live pipeline against real Gemini, market-data, news, and SEC services. Such testing would require live credentials, controlled external-service availability, and more complex infrastructure coordination. For this reason, the current automated suite should be understood as strong module and API verification, supplemented by limited manual system checks, rather than as complete production-grade end-to-end automation.

This scope is appropriate for the current academic phase of the project, but it also defines a clear improvement opportunity for future work.

6.5 Test Cases and Bug Reports

A formal test-case representation is required by the university format. Rather than listing every single assertion in the repository, the chapter uses representative test cases that show the kinds of behaviors validated by the automated suite.

Table 6.4 here: Representative test case table.

Test Case ID	Input	Expected Output	Actual Output	Status
TC-U-01	Create job with query and ticker	Pending job record	Pending record returned	PASS
TC-U-10	Mid-sentence narrative text	Trim to complete sentence	Trimmed correctly	PASS
TC-U-18	Valid report artifact	PDF file created	Non-empty PDF created	PASS
TC-U-24	Gemini error during resolve	Fallback ticker, no crash	Fallback executed	PASS
TC-I-04	POST /reports valid body	202 + job id	202 returned with id	PASS
TC-I-05	POST /reports empty query	422 validation error	422 returned	PASS

In addition to the formal tests, defect identification and correction were an important part of quality assurance. Testing and manual verification exposed real issues in the implementation, and these were addressed before the report-writing phase advanced.

Table 6.5 here: Bug report and fix summary.

Bug ID	Symptom	Root Cause	Fix Applied	Verification
BUG-01	Ticker resolution failed on AI exception	Exception path not fully caught	Expanded fallback handling around <code>generate_json</code> call	TC-U-24 pass
BUG-02	Narrative ended mid-sentence	Output finalization missing strict sentence trim	Added <code>finalize/trim</code> cleanup logic	TC-U-10 pass
BUG-03	PDF cover visual artifact	Cover rendering style issue	Revised cover rendering with safer layout approach	PDF tests + manual check

One important bug involved ticker resolution. Earlier in development, an exception thrown by the AI client during `generate_json` could escape the resolution flow instead of being handled by fallback logic. The symptom was that report generation could fail at the ticker-resolution stage when the external AI response path failed unexpectedly. The root cause was incomplete exception handling around the resolution call. The fix was to expand the protected code path so that errors from the AI call itself were caught and a fallback ticker could still be returned. TC-U-24 now verifies this behavior and acts as a regression test.

A second quality issue involved narrative truncation. In some cases, generated text could end visibly in the middle of a sentence, reducing report readability. The fix introduced narrative-finalization logic that trims text back to the last complete sentence ending and normalizes formatting. This behavior is now validated by tests such as TC-U-10.

A third defect affected PDF presentation. Earlier report output included a cover-page rendering issue that produced an undesirable dark block instead of the intended styled cover layout. The PDF generation logic was revised to render the cover using a safer styled table approach. Subsequent PDF tests and manual PDF inspection confirmed that document creation remained stable after the fix.

These examples are useful because they show that testing was not performed merely as a documentation exercise. It directly improved the robustness, readability, and usability of the final system.

6.6 Testing Limitations

Although the testing work completed for this project is meaningful, it is important to document its current limits honestly. First, external dependencies are heavily mocked in

the automated suite. This means that Gemini, market-data providers, news feeds, and SEC retrieval are not exercised live during routine test execution. This is beneficial for repeatability and offline execution, but it also means that external-service outages, schema changes, or unpredictable live responses are not fully covered by the automated suite.

Second, the project does not yet include frontend automation for the Streamlit interface. User-interface behaviors such as browser interaction, progressive event rendering, and download-button behavior are therefore not verified through tools such as Selenium or Playwright. These behaviors were checked only through manual use during development.

Third, the current implementation uses an in-memory job store rather than a persistent database. As a result, there are no persistence, migration, or database-integrity tests to discuss in this chapter. Fourth, the project does not yet include a dedicated performance, load, or stress test suite, so the observed behavior under heavy concurrent usage cannot be stated as formally validated. Finally, although `pytest-cov` is configured as part of the development toolchain, a formal coverage percentage is not included in this chapter because a dedicated coverage capture was not recorded as part of the validated evidence set.

These limitations do not invalidate the testing work; rather, they define the current verification boundary of the project and provide a clear roadmap for future testing enhancement.

6.7 Chapter Summary

This chapter has described the testing and quality-assurance work performed for the AI Stock Report system after implementation. The project uses a layered verification strategy consisting of unit testing for critical logic slices, integration testing for FastAPI route behavior, and limited manual system verification for the integrated backend and frontend workflow. In the validated test run, all 33 tests passed, providing concrete evidence that the main modules and API behaviors operate correctly within the scope of the current architecture.

The chapter also showed that testing contributed directly to software improvement. Real defects such as ticker-resolution failure handling, narrative truncation, and PDF-rendering issues were identified and corrected, with regression-style tests helping to prevent recurrence. This demonstrates that testing was not treated as a formality, but as an active part of engineering quality.

At the same time, the discussion remained realistic about the current scope of verification. External services are mocked for repeatability, frontend automation is not yet implemented, the in-memory architecture limits persistence-related testing, and no formal performance suite has yet been executed. These limitations should be viewed as future extension points rather than hidden weaknesses.

Overall, the testing work provides sufficient evidence that the implemented system is stable enough for academic demonstration and structured evaluation. Having established how correctness was verified, the next chapter can move to results and discussion, where actual outputs, observations, and achieved improvements can be analysed in more detail.

Chapter 7: Results and Discussion

7.1 Introduction

This chapter presents the practical results achieved by the AI Stock Report system and discusses what those results mean in relation to the project objectives, the earlier literature review, and the completed implementation and testing work. If Chapter 5 explained how the system was built and Chapter 6 explained how it was verified, this chapter explains what the finished system actually produces, how it behaves from a user perspective, and what value it delivers as an integrated software solution.

The purpose of the chapter is not to repeat implementation details or restate every test case. Instead, it focuses on observable outcomes. These outcomes include the user-facing workflow, the generated PDF artifact, the visibility of progress during report generation, the qualitative performance characteristics of the pipeline, and the extent to which the proposed system improves on more fragmented or manual approaches to stock research. The chapter also compares the final system with the existing systems discussed in Chapter 2 and evaluates the improvements achieved by the project.

The discussion is intentionally balanced. The project demonstrates a successful end-to-end stock-report generation workflow that combines natural-language interaction, automated evidence gathering, chart production, AI-generated narrative, and formal PDF output. At the same time, the chapter remains realistic about the current scope of the system. The application should be understood as a validated academic prototype and MVP with deployment readiness, not as a complete replacement for professional institutional research platforms.

7.2 Output Screenshots and Observed Results

The most immediate result of the project is that a user can move from a plain-English stock request to a structured downloadable report through a single integrated workflow. Instead of manually switching between finance portals, spreadsheets, news pages, regulatory sources, and writing tools, the user interacts with one interface and receives one output artifact. This is the main practical result achieved by the project.

Table 7.1 here: Observable system outputs and meaning.

Observed Output	What It Demonstrates
Query input screen	Usable entry point for non-technical users
Live progress timeline	Transparent long-running workflow
Download-ready completion state	End-to-end artifact generation success
API health/docs page	Service availability and maintainability
Generated PDF cover and pages	Structured reporting artifact quality

The first visible result is the Streamlit home screen, where the user enters a natural-language request such as asking for a report on a particular company. The presence of an optional ticker input is also important because it supports both non-expert users who prefer natural language and more experienced users who already know the relevant stock symbol. This confirms that the system is accessible at the point of entry.

The second important output is the live progress timeline. As the backend resolves the ticker, gathers research, writes sections, and prepares the PDF, progress events are streamed to the frontend through server-sent events. This result matters because long-running AI and data-processing tasks can otherwise appear unresponsive. The visible timeline improves transparency and makes the workflow easier to trust and interpret.

Once generation is complete, the system produces a download-ready state in the frontend. This is one of the clearest indicators that the workflow is successful, because the user is not left with intermediate fragments or raw JSON responses. Instead, the output is a coherent artifact that can be downloaded, stored, shared, or included in later review.

A further result is the visible availability of the backend service itself. The health endpoint and API documentation page demonstrate that the backend is exposed as a usable service rather than existing only as embedded logic behind the frontend. This is relevant both for development and for operational discussion, because it shows that the system is service-oriented rather than a single local script.

The generated PDF is the main result artifact of the project. Its cover page, structured sections, embedded charts, and narrative layout demonstrate that the system produces more than a data dump. It produces a report-like deliverable suitable for reading and review. The presence of visual charts inside the report is especially important because it confirms that the system successfully combines textual interpretation with numerical and graphical evidence.

Overall, the observed results show that the project achieves a complete workflow from input query to polished report output. This is the strongest practical outcome of the implemented system.

7.3 Performance Evaluation

The performance of the AI Stock Report system should be discussed in qualitative rather than purely numerical terms, because the current evidence set does not include a formal benchmark or load-testing campaign. Even so, several important performance-related observations can be made from the implemented workflow and validated system behavior.

Dimension	Observation	Interpretation
Research phase	Parallel retrieval reduces serial waiting	Better responsiveness
User feedback	SSE progress continuously updates UI	Improved perceived performance
Pipeline completion	Full flow reaches downloadable artifact	Functional adequacy for MVP
Latency drivers	External APIs + LLM dominate time	Expected dependency-bound behavior
Load evidence	No formal stress benchmark yet	Future testing requirement

The first important observation is that the architecture reduces unnecessary blocking through asynchronous orchestration and parallel research collection. Market data, fundamentals, news, and SEC filing retrieval are launched concurrently rather than strictly one after another. This design improves the overall responsiveness of the research phase, especially when external sources have different response times.

The second important observation is that perceived performance is improved by the use of server-sent events. Even when the total report-generation time is influenced by external APIs and language-model response latency, the user is able to observe progress in real time. This is a practical performance improvement from the user's perspective because the system remains communicative during a long-running task instead of appearing frozen.

A third observation is that the output pipeline remains coherent across multiple stages. The system not only gathers data, but also generates charts, writes narrative sections, assembles a PDF, stores the report path, and exposes the completed artifact for download. The successful completion of this multi-stage pipeline indicates functional performance adequacy for the intended academic and MVP use case.

At the same time, the evaluation must remain realistic. The dominant sources of latency in the system are likely to be external data retrieval and Gemini-based text generation rather than local Python control flow. This means that the architecture can improve coordination and responsiveness, but cannot eliminate all waiting time. In addition, no formal throughput, concurrency, or stress-test results are available in the current evidence set. For that reason, the system can be described as operationally responsive and user-transparent in controlled usage, but not yet formally benchmarked for heavy production traffic.

7.4 Comparison with Existing Systems

A useful discussion of results must relate the final system back to the landscape examined in Chapter 2. The key question is not whether the proposed application outperforms every existing financial platform, but whether it addresses the specific workflow gap identified earlier.

Table 7.3 here: Comparison with existing systems.

Criterion	Proposed System	Bloomberg	Yahoo Finance	OpenBB	General AI Assistant
Accessibility	High	Low	High	Medium	High
Cost	Low	High	Low	Low-Medium	Medium
End-to-end automation	High	Medium	Low	Medium	Low
Built-in report artifact	Yes (PDF)	No direct custom auto-PDF	No	Not primary focus	No
Progress visibility	Yes	Limited	No	Limited	No

Compared with Bloomberg Terminal, the proposed system is far narrower in scope but significantly more accessible. Bloomberg offers a much richer professional ecosystem, deeper analytics, and enterprise-grade market coverage. However, it is expensive and still relies heavily on skilled user interaction inside a complex environment. The proposed system does not compete on market breadth or institutional sophistication. Its advantage lies in automating a specific reporting workflow at much lower cost and complexity.

Compared with Yahoo Finance, the proposed system shows a clearer automation result. Yahoo Finance is widely accessible and useful for manual browsing of prices, company information, and news. However, it does not convert a user request into a generated research document. The current application improves on this by automating ticker resolution, multi-source collection, chart creation, and report assembly into one process.

Compared with OpenBB, the proposed system offers a different type of value. OpenBB is powerful for technically skilled users who want modular, open financial tooling. Its strength is extensibility and developer-oriented analysis. The proposed system is narrower, but more workflow-focused. It is designed specifically to produce a structured

stock report with live progress feedback and downloadable PDF output rather than to serve as a general-purpose financial analysis environment.

Compared with general-purpose AI assistants, the project demonstrates stronger workflow integration. A standalone AI assistant can generate fluent prose, but it does not by itself guarantee structured access to price history, fundamentals, news, SEC filings, chart generation, or artifact delivery. The proposed system improves on this by placing AI-generated narrative inside an evidence-backed pipeline with formal report output.

These comparisons show that the project’s contribution is not maximum feature count. Its contribution is workflow integration: bringing together accessible data sources, automation, charting, language generation, progress visibility, and PDF delivery in a single process.

7.5 Improvements Achieved

The implemented system achieves several concrete improvements over fragmented and manual stock-research workflows.

Table 7.4 here: Improvements achieved by the proposed system.

Improvement Area	Before (Manual Workflow)	After (Proposed System)
Ticker and context setup	Manual symbol lookup	Automated resolution with optional override
Evidence gathering	Multi-site manual collection	Parallel integrated retrieval
Report writing	Manual synthesis effort	AI-assisted section generation
Visual analysis	Separate charting steps	Built-in chart generation
Output delivery	Ad hoc notes/doc creation	Structured downloadable PDF
User transparency	Opaque waiting periods	Live progress timeline

The first major improvement is the reduction of manual effort. In a traditional workflow, a user must identify the ticker, inspect multiple data sources, review recent news, locate filings, generate their own charts, and then write a coherent report manually. The

proposed system automates much of this process, turning a loosely specified request into a structured research artifact.

The second improvement is workflow consolidation. Instead of switching between separate websites, tools, and notes, the user interacts with a single frontend connected to a single orchestrated backend pipeline. This reduces fragmentation and improves usability.

The third improvement is output structure. Many free finance tools provide data, but not a formal report. Many AI tools provide narrative, but not a grounded document with charts and section structure. The current system improves on both extremes by combining evidence gathering and narrative generation in a PDF-based report format.

The fourth improvement is transparency during execution. Because progress events are streamed during processing, the user can see whether the system is identifying the ticker, collecting research, writing sections, or completing the final artifact. This makes the system easier to understand and more trustworthy than a silent long-running request.

The fifth improvement is operational readiness. The project is not limited to one local script or notebook. It includes API and frontend separation, Docker support, and deployment-oriented packaging. This strengthens the practical value of the project as a software system rather than only as a proof-of-concept demonstration.

7.6 Discussion of Findings

The results of the project show that the core research gap identified earlier can be addressed effectively through an integrated agentic workflow. The system demonstrates that it is feasible to accept a natural-language stock request, resolve the relevant company context, gather multiple forms of financial evidence, generate charts, produce structured narrative sections, and return a downloadable report artifact through one coordinated application.

This is a meaningful result because the project does not rely on only one strength. It combines backend orchestration, asynchronous execution, user-facing progress visibility, AI-assisted writing, and formal document output. The value of the system therefore lies in integration rather than in any single isolated feature.

At the same time, the results also highlight important constraints. The quality and timeliness of output remain dependent on external services such as Gemini and public financial data providers. The current system is also better understood as an MVP and academic prototype than as a fully hardened enterprise platform. It has strong evidence for correctness in core modules and route behavior, but it does not yet include full automated live end-to-end validation, frontend automation, or formal performance benchmarking.

Even with these limits, the findings remain positive. The project has moved beyond a conceptual design and beyond a collection of disconnected tools. It demonstrates a usable, tested, and deployment-ready workflow that transforms stock research into a more automated and structured process. In the context of an MSc Data Science project, this is a significant and defensible outcome.

7.7 Chapter Summary

This chapter presented the practical results of the AI Stock Report system and discussed what those results show about the value of the project. The system successfully produces a user-facing workflow that starts with a natural-language query and ends with a downloadable PDF report containing structured narrative sections and charts. The visible frontend flow, live progress timeline, backend service availability, and final PDF artifact together show that the system operates as an integrated application rather than as a theoretical design.

The chapter also showed that the system compares favourably with more fragmented alternatives in the specific area it was designed to address. It is more accessible than premium institutional terminals, more automated than free finance portals, more workflow-focused than general open financial platforms, and more grounded than a standalone AI assistant. Its main contribution is therefore the integration of multiple capabilities into one coherent reporting pipeline.

The discussion remained realistic about the current boundaries of the work. The project demonstrates strong practical value as an academic prototype and MVP, but it is not yet positioned as a complete enterprise-grade replacement for professional financial-research platforms. This balanced interpretation of results provides an appropriate bridge to the final chapter, which will conclude the project and identify future enhancements.

Chapter 8: Conclusion and Future Enhancements

8.1 Introduction to the Final Chapter

This chapter concludes the AI Stock Report project by evaluating the final system against the original objectives, summarizing the most important achievements, presenting the technical learning outcomes, and outlining specific future enhancements. The aim is to provide a clear end-state assessment based on the completed implementation, testing, and results discussion.

The project started with a practical problem: stock research is information-heavy, fragmented across multiple tools, and difficult to convert into a structured report quickly. The proposed solution was an integrated agentic workflow that accepts a natural-language query, gathers multi-source evidence, generates section-wise narrative and charts, and returns a downloadable PDF report with live progress visibility. Chapters 5, 6, and 7 demonstrated that this workflow was built, validated, and observed in practice. This final chapter therefore consolidates those outcomes and defines how the system can evolve beyond its current MVP scope.

8.2 Conclusion: Achievement of Objectives

The primary objective of the project was to implement a working system that automates the generation of an equity research report from a natural-language request. This objective has been achieved. The completed system accepts user queries through a Streamlit interface, resolves ticker context, executes a multi-stage backend pipeline, and produces a structured PDF report as the final artifact.

A second objective was to gather diverse evidence types required for meaningful stock analysis, including market prices, fundamentals, news, and regulatory filings. This objective was also achieved through the research workflow that integrates financial and textual evidence sources before report writing begins.

A third objective was to provide transparent progress updates during long-running report generation. This objective was achieved through server-sent events that stream stage-wise progress to the frontend. From a user perspective, this improved trust and usability because the process remains visible rather than opaque.

A fourth objective was to produce a report that combines narrative analysis and visual evidence. This objective was achieved through chart generation and section-based narrative composition, followed by PDF assembly using a formal report structure.

A fifth objective was to maintain modular architecture and implementation clarity. This objective was achieved by separating concerns across routing, orchestration, research, report generation, charting, and PDF services, which improved maintainability and traceability.

A sixth objective was to validate system behavior through testing. This objective was achieved through a completed pytest suite covering unit and integration layers, with all tests passing in the validated run and defect fixes incorporated through regression-aware checks.

A seventh objective was to prepare the system for practical execution beyond a single local script. This objective was achieved through containerization support and deployment-oriented artifacts, which establish readiness for broader operational environments.

Table 8.1 here: Objective achievement matrix (Objective, Evidence, Status).

Objective	Evidence	Status
Natural-language query to report pipeline	Streamlit + orchestrator + PDF artifact	Achieved
Multi-source evidence collection	Research agent tools	Achieved
Live progress visibility	SSE events and UI timeline	Achieved
Structured report output with charts	Report agent + chart builder + PDF generator	Achieved
Quality validation through tests	33 passing tests (unit + integration)	Achieved
Deployment readiness	Docker + CI/CD artifacts	Achieved
Production-grade hardening	Auth, persistence, performance benchmarks	Partial / Future scope

Taken together, these outcomes indicate that the project met its intended objectives within the defined scope. The current system should be interpreted as a validated academic prototype and MVP that demonstrates full workflow feasibility rather than as a finished enterprise platform.

8.3 Key Achievements

The project delivered several significant achievements that are both technically meaningful and practically relevant.

The first key achievement is the successful implementation of an end-to-end agentic report-generation pipeline. The system does not stop at data retrieval or narrative generation in isolation; it completes the full journey from user query to downloadable report artifact.

The second key achievement is integration across heterogeneous tasks that are often disconnected in real workflows. Ticker identification, financial evidence retrieval, news and filing context collection, chart rendering, AI narrative generation, and PDF composition were brought together into a single orchestrated process.

The third key achievement is live operational transparency through progress streaming. This is an important usability improvement for AI-assisted workflows because it keeps users informed during asynchronous and externally dependent processing stages.

The fourth key achievement is quality-focused maturation of the implementation through testing and bug resolution. Real issues were identified and corrected, including robustness in ticker fallback handling, narrative cleanup behavior, and PDF presentation quality.

The fifth key achievement is deployment-oriented maturity. The project includes clear packaging and environment strategies that support repeatable execution in local and containerized contexts, moving the work beyond a one-off prototype script.

Collectively, these achievements show that the project is implementation-rich and outcome-oriented, consistent with MSc (Data Science) expectations.

8.4 Technical Learning Outcomes

The project produced strong technical learning outcomes across software engineering, data integration, and AI-enabled application design.

One major learning outcome was the practical use of asynchronous orchestration in a multi-stage workflow. Designing and coordinating concurrent research tasks while preserving traceable progress events required careful control-flow thinking and clear state handling.

A second learning outcome was structured integration of external services. The project required working with heterogeneous data providers and handling partial failures gracefully, reinforcing the importance of resilient engineering when external dependencies are unavoidable.

A third learning outcome was controlled use of generative AI within a deterministic pipeline. Rather than treating LLM output as free-form text generation, the project demonstrated how AI can be constrained by evidence context, section-level instructions, and post-processing logic to improve report quality.

A fourth learning outcome was the value of testing as an active engineering mechanism. Unit and integration testing were not peripheral; they directly improved implementation reliability and helped detect defects that were not obvious from static inspection.

A fifth learning outcome was deployment awareness. Building with container readiness and environment-driven configuration from early stages improves reproducibility and reduces friction when moving from development to operational contexts.

These learning outcomes are important because they reflect practical system-building skills, not just conceptual understanding.

8.5 Future Enhancements

Although the current system satisfies its defined MVP objectives, several concrete enhancements can significantly increase capability, robustness, and user value.

The first enhancement is persistent data and job storage. The current in-memory job state is suitable for MVP operation, but a production-oriented version should introduce persistent storage for job history, report metadata, and audit trails.

The second enhancement is authentication and access control. Adding user accounts and role-based authorization would improve security and make the platform suitable for multi-user operation.

The third enhancement is stronger external-dependency reliability controls. Retry policies, circuit-breaker behavior, fallback strategies, and more explicit external-source

health signaling would improve resilience when upstream services are slow or unavailable.

The fourth enhancement is formal performance and load evaluation. Controlled benchmark scenarios should be added to quantify response-time behavior, concurrent-request handling, and throughput characteristics under heavier workloads.

The fifth enhancement is frontend automation and UX evolution. Automated UI testing, richer status visualization, historical report browsing, and better report filtering can improve usability and maintainability.

The sixth enhancement is mobile access. A mobile-friendly interface or dedicated client would improve accessibility for users who need quick report workflows outside desktop contexts.

The seventh enhancement is deeper cloud production architecture. Future work can include managed persistence layers, scalable worker design, centralized observability, and stronger operational monitoring for reliable multi-environment deployment.

The eighth enhancement is advanced AI capability expansion. Future iterations can support cross-company comparative reports, portfolio-level synthesis, scenario-based summaries, and user-configurable report depth while preserving evidence grounding.

These enhancements are realistic next steps that build directly on the current architecture and can be prioritized incrementally.

8.6 Final Closure

The AI Stock Report project successfully demonstrates that an integrated, agentic workflow can transform a natural-language stock request into a structured, evidence-backed, downloadable research report. The system achieved its main objectives,

delivered meaningful technical outcomes, and validated core behavior through systematic testing and observed results.

At the same time, the project has clearly identified where future work should focus to move from validated MVP toward broader production maturity. This combination of delivered value and actionable extension path provides a strong and complete conclusion for the MSc (Data Science) project.

Chapter 9: References

This chapter follows APA style and includes credible sources only (official documentation, standards, peer-reviewed research papers, and authoritative books/publishers).

1. Amazon Web Services. (n.d.). *Amazon Elastic Container Service documentation*. Retrieved May 30, 2026, from <https://docs.aws.amazon.com/ecs/>
2. Bodie, Z., Kane, A., & Marcus, A. J. (2021). *Investments* (12th ed.). McGraw-Hill Education.
3. Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N., Chen, A., Creel, K., Davis, J. Q., Demszky, D., ... Liang, P. (2021). On the opportunities and risks of foundation models. *arXiv*. <https://arxiv.org/abs/2108.07258>
4. Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33*, 1877-1901. <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
5. Docker, Inc. (n.d.). *Docker documentation*. Retrieved May 30, 2026, from <https://docs.docker.com/>
6. Docker, Inc. (n.d.). *Docker Compose documentation*. Retrieved May 30, 2026, from <https://docs.docker.com/compose/>

7. Feedparser Project. (n.d.). *Universal Feed Parser documentation*. Retrieved May 30, 2026, from <https://feedparser.readthedocs.io/>
8. Fielding, R. T., & Reschke, J. (2022). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and content (RFC 9110)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
9. GitHub, Inc. (n.d.). *GitHub Actions documentation*. Retrieved May 30, 2026, from <https://docs.github.com/actions>
10. Graham, B., & Dodd, D. L. (2008). *Security analysis: Principles and technique* (6th ed.). McGraw-Hill Education.
11. Matplotlib Development Team. (n.d.). *Matplotlib documentation*. Retrieved May 30, 2026, from <https://matplotlib.org/stable/>
12. Pydantic Services Inc. (n.d.). *Pydantic documentation*. Retrieved May 30, 2026, from <https://docs.pydantic.dev/>
13. Pydantic Services Inc. (n.d.). *pydantic-settings documentation*. Retrieved May 30, 2026, from https://docs.pydantic.dev/latest/concepts/pydantic_settings/
14. Python Software Foundation. (n.d.). *Python 3 documentation*. Retrieved May 30, 2026, from <https://docs.python.org/3/>
15. pytest-dev. (n.d.). *pytest documentation*. Retrieved May 30, 2026, from <https://docs.pytest.org/>

16. ReportLab Inc. (n.d.). *ReportLab user guide*. Retrieved May 30, 2026, from <https://www.reportlab.com/documentation/>
17. Securities and Exchange Commission. (n.d.). *SEC EDGAR API and data documentation*. Retrieved May 30, 2026, from <https://www.sec.gov/search-filings/edgar-search-assistance/accessing-edgar-data>
18. Streamlit Inc. (n.d.). *Streamlit documentation*. Retrieved May 30, 2026, from <https://docs.streamlit.io/>
19. The WHATWG HTML Standard. (n.d.). *Server-sent events (EventSource)*. Retrieved May 30, 2026, from <https://html.spec.whatwg.org/multipage/server-sent-events.html>
20. Uvicorn Contributors. (n.d.). *Uvicorn documentation*. Retrieved May 30, 2026, from <https://www.uvicorn.org/>
21. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
22. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems, 35*, 24824-24837. https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html
23. Yahoo Finance API Community (ranaroussi). (n.d.). *yfinance documentation and repository*. GitHub. Retrieved May 30, 2026, from <https://github.com/ranaroussi/yfinance>

24. Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2021). Understanding deep learning (still) requires rethinking generalization. **Communications of the ACM*, 64*(3), 107-115. <https://doi.org/10.1145/3446776>
25. Klopfenstein, L. C., Delpriori, S., Malatini, S., & Bogliolo, A. (2017). The rise of bots: A survey of conversational interfaces, patterns, and paradigms. In **Proceedings of the 2017 Conference on Designing Interactive Systems** (pp. 555-565). ACM. <https://doi.org/10.1145/3064663.3064672>
26. OpenAI. (2023). **GPT-4 technical report**. **arXiv**. <https://arxiv.org/abs/2303.08774>
27. Microsoft. (n.d.). **Visual Studio Code documentation**. Retrieved May 30, 2026, from <https://code.visualstudio.com/docs>
28. Google. (n.d.). **Google AI for Developers documentation (Gemini API)**. Retrieved May 30, 2026, from <https://ai.google.dev/>

Chapter 10: Appendices

Appendix A: Complete Source Code Index (Project Map)

The complete source code for this project is provided in the project repository and submission package. This appendix gives a navigational index so that examiners can quickly locate each implementation area without scanning all files manually.

A.1 Root-Level Files

- ``pyproject.toml``: Python version, dependencies, scripts, and testing configuration.
- ``README.md``: Architecture overview, setup instructions, runtime flow, and API usage examples.
- ``streamlit_app.py``: Streamlit frontend implementation.
- ``Dockerfile``: API service container build definition.
- ``Dockerfile.streamlit``: Streamlit service container build definition.
- ``docker-compose.yml``: Local multi-service composition.

A.2 Backend Application Package (``app/``)

- ``app/main.py``: FastAPI application entry point and health route.
- ``app/config.py``: Environment-driven settings model.
- ``app/schemas.py``: Request/response/progress schemas.
- ``app/dependencies.py``: Dependency providers.
- ``app/orchestrator.py``: Core report pipeline orchestration.

A.3 Agents (``app/agents/``)

- ``app/agents/base.py``: Agent base abstraction.
- ``app/agents/research.py``: Parallel evidence collection workflow.
- ``app/agents/report.py``: Section writing workflow and report assembly trigger.

A.4 Services (`app/services/`)

- `app/services/gemini\_client.py`: Gemini API wrapper for text/JSON generation.
- `app/services/job\_store.py`: In-memory async job state manager.
- `app/services/chart\_builder.py`: Chart generation to PNG bytes.
- `app/services/pdf\_generator.py`: ReportLab PDF generation.

A.5 Tools (`app/tools/`)

- `app/tools/ticker\_resolve.py`: Ticker/company resolution from query.
- `app/tools/market\_data.py`: Market-price retrieval and technical summary.
- `app/tools/fundamentals.py`: Company fundamentals and valuation retrieval.
- `app/tools/news.py`: News retrieval via RSS parsing.
- `app/tools/sec\_filings.py`: SEC filings retrieval.

A.6 Models (`app/models/`)

- `app/models/stock.py`: Stock-research domain models.
- `app/models/report.py`: Report artifact and section models.

A.7 Routers (`app/routers/`)

- `app/routers/reports.py`: Report lifecycle routes (create/stream/status/download).

A.8 Tests (`tests/`)

- `tests/unit/test\_job\_store.py`
- `tests/unit/test\_narrative\_finalize.py`
- `tests/unit/test\_chart\_builder.py`
- `tests/unit/test\_pdf\_generator.py`
- `tests/unit/test\_ticker\_resolve.py`
- `tests/integration/test\_api.py`

- ``tests/conftest.py``

These files represent the complete code index used for implementation, validation, and deployment preparation.

Appendix B: Database / Data Storage Note and Runtime Data Model

The project requirement template mentions including a database dump where applicable. In the current implementation, there is no relational database layer and therefore no SQL dump file to provide. This is an intentional MVP design decision.

B.1 Current Storage Approach

- The system uses an in-memory asynchronous JobStore to hold runtime job state.
- Job state includes: job ID, query, ticker, company name, status, events, PDF path, and optional error state.
- Generated PDF files are persisted to the reports directory as filesystem artifacts.

B.2 Why No Relational Dump Is Included

- The present scope targets a validated single-instance workflow prototype.
- Persistent multi-user history and relational reporting storage are future enhancements.
- Therefore, a relational schema dump is not applicable in this version.

B.3 Conceptual Runtime Data Entities

- ``ReportRequest``: incoming query and optional ticker.
- ``JobRecord``: runtime processing state.
- ``StockResearch``: consolidated evidence bundle.
- ``ReportSection``: section title, narrative, optional chart bytes.
- ``ReportArtifact``: final report metadata plus section collection.

This satisfies the requirement to explain data organization while remaining faithful to the implemented architecture.

Appendix C: User Manual

C.1 Purpose

This manual explains how an end user can generate and download a stock report using the application.

C.2 User Workflow

1. Open the Streamlit UI.
2. Enter a natural-language query (for example, request a report for a specific company).
3. Optionally enter a ticker if known.
4. Submit the request.
5. Observe live progress updates while research and report generation run.
6. Download the generated PDF when completion is shown.

C.3 Input Guidance

- Use clear company names or stock intents in the query.
- If ticker is known, optional ticker input can speed ticker-resolution stage.
- Keep query meaningful (non-empty input required).

C.4 Output Interpretation

The resulting PDF typically contains:

- Executive summary
- Company overview
- Price and financial performance discussion
- Margin and valuation sections
- News and SEC filing context
- Risks and investment thesis
- Embedded charts where applicable

C.5 Common User Issues

- Missing API key configuration can block AI narrative generation.
- Temporary external-service unavailability can delay or reduce content depth.
- If generation fails, review backend logs and environment configuration.

Appendix D: Installation Guide

D.1 Prerequisites

- Python 3.12+
- `uv` package manager
- Valid Gemini API key

D.2 Local Setup Steps

1. Clone the repository.
2. Create `.env` from `.env.example`.
3. Set required environment variables (especially Gemini key).
4. Install dependencies:

```
```bash
```

```
uv sync
```

```
```
```

5. Start backend API:

```
```bash
```

```
uv run ai-stock-report-api
```

```
```
```

6. Start Streamlit frontend in another terminal:

```
```bash
```

```
uv run streamlit run streamlit_app.py
```

```
```
```

7. Open UI at `http://localhost:8501`.

D.3 Optional Containerized Setup

```
```bash
```

```
docker compose up --build
```

```
```
```

This starts API and Streamlit services with shared report storage volume.

D.4 Validation Checks

- API health endpoint responds successfully.
- Streamlit interface loads in browser.
- A sample report can be generated and downloaded.

Appendix E: API Usage Quick Reference

E.1 Endpoints

- ``GET /health``: service health check.
- ``GET /reports/stream?query=...``: starts report generation and streams progress events.
- ``GET /reports/{job_id}/status``: report status check.
- ``GET /reports/{job_id}/download``: download generated report.

E.2 Example Stream Request

```
```bash
curl -N
"http://localhost:8000/reports/stream?query=Generate+a+report+for+Nvidia+stock"
```
```

E.3 Example Download Request

```
```bash
curl -o report.pdf "http://localhost:8000/reports/{job_id}/download"
```
```

Appendix F: Sample Output and Screenshot Placeholders

Insert the following screenshot artifacts in final Word/PDF assembly.

- Screenshot F.1 here: Streamlit home screen with query input.
- Screenshot F.2 here: Live progress timeline while generation is running.
- Screenshot F.3 here: Completion state with download button.
- Screenshot F.4 here: FastAPI health endpoint or docs page.
- Screenshot F.5 here: Generated PDF cover page.
- Screenshot F.6 here: Internal PDF page with chart and narrative section.

Appendix G: Testing Execution Evidence (Optional Supporting Appendix)

Representative command used for validation:

```
```bash
.venv\Scripts\pytest.exe tests/ -v --tb=short
```
```

Observed outcome in validated run:

- All tests passed.
- Unit and integration suites executed successfully.

This appendix supports Chapter 6 without duplicating full test-case discussion.

Appendix H: Submission Packaging Checklist

- Full source code included in submission package.
- Chapter files included and formatted.

List of Abbreviations

| Abbreviation | Expanded Form |
|--------------|--|
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| AWS | Amazon Web Services |
| CI/CD | Continuous Integration / Continuous Deployment |
| DFD | Data Flow Diagram |
| ECS | Elastic Container Service |
| EDGAR | Electronic Data Gathering, Analysis, and Retrieval |
| HTTP | Hypertext Transfer Protocol |
| LLM | Large Language Model |
| MA50 / MA200 | 50-day / 200-day Moving Average |
| MVP | Minimum Viable Product |
| PDF | Portable Document Format |
| P/E | Price-to-Earnings Ratio |
| P/S | Price-to-Sales Ratio |
| REST | Representational State Transfer |
| RSS | Really Simple Syndication |
| SEC | U.S. Securities and Exchange Commission |
| SSE | Server-Sent Events |
| UI/UX | User Interface / User Experience |
| UV | Python Package and Environment Manager |